Software Requirements Specification

Phase 1 : Project 1

Student Grade Calculator

Project Type: Java Console Application
Difficulty Level: ⭐ Beginner
Programming Language: Java

---

1. Project Overview

1.1 Introduction

The **Student Grade Calculator** is a Java console-based application designed to automate the process of calculating a student's academic performance. The application accepts marks obtained in multiple subjects, calculates the total marks, average, and percentage, assigns an appropriate grade, determines whether the student has passed or failed, and calculates the student's rank among all entered students.

This project provides beginners with an opportunity to apply fundamental Java programming concepts while solving a practical real-world problem.

1.2 Purpose

The purpose of this project is to develop a simple and reliable application that eliminates manual grade calculations and provides accurate academic results automatically. It also serves as a practical learning exercise for students beginning their Java programming journey.

1.3 Objectives

The objectives of this project are to:

- Collect student information and subject marks.
- Calculate total marks, average, and percentage.
- Determine grades based on predefined grading criteria.

- Identify pass or fail status.
- Calculate student rankings based on academic performance.
- Display results in a clear and organized report format.
- Strengthen understanding of Java programming fundamentals.

1.4 Scope

The application is limited to a console-based environment and stores data only while the program is running. It focuses on academic result processing without using databases or graphical interfaces.

The system includes:

- Student information management
- Marks entry
- Result calculation
- Grade assignment
- Pass/Fail determination
- Student ranking
- Report generation

1.5 Real-World Applications

Although simplified for learning purposes, the concepts used in this project are similar to those found in:

- School result processing systems
- College examination portals
- Online grading systems
- Student management software
- Academic performance analysis tools

---

2. Problem Statement

Educational institutions often calculate student results manually or using spreadsheets. Manual calculation becomes difficult when handling multiple students and subjects. It increases the possibility of calculation mistakes, consumes valuable time, and makes ranking students more complicated.

The Student Grade Calculator addresses these issues by automatically processing student marks and producing accurate academic results.

The application helps users by:

- Eliminating calculation errors
- Reducing processing time
- Providing consistent grading
- Automatically determining pass/fail status
- Generating rankings based on performance

Expected Users

- Students learning Java programming
- Teachers
- Educational institutions
- Beginners practicing programming logic

---

3. Features

3.1 Student Information Input

The system accepts basic student details such as:

- Student ID
- Student Name
- Number of Subjects

3.2 Subject Marks Input

Users enter marks for each subject individually.

The system validates every mark before storing it.

3.3 Total Marks Calculation

The application calculates the sum of marks obtained in all subjects.

3.4 Percentage Calculation

The system calculates the percentage using the formula:

Percentage = (Total Marks / Maximum Marks) × 100

3.5 Average Calculation

The application computes the average marks scored per subject.

3.6 Grade Assignment

Grades are assigned according to the student's percentage.

Example grading table:

Percentage	Grade
90–100	A+
80–89	A
70–79	B+
60–69	B
50–59	C
35–49	D
Below 35	F

3.7 Pass/Fail Determination

A student is considered **Pass** only if:

- Every subject mark is **35 or above**.

Otherwise, the student is declared **Fail**.

3.8 Student Ranking

After processing all students, the application assigns ranks based on percentage.

Higher percentage receives a better rank.

3.9 Result Display

The application displays:

- Student ID
- Student Name
- Subject Marks
- Total Marks
- Average
- Percentage
- Grade
- Pass/Fail Status
- Rank

---

4. Functional Requirements

The system shall:

FR-01 Accept student information.

FR-02 Accept marks for multiple subjects.

FR-03 Validate marks before storing.

FR-04 Calculate total marks.

FR-05 Calculate average marks.

FR-06 Calculate percentage.

FR-07 Assign grades.

FR-08 Determine pass or fail status.

FR-09 Calculate student rankings.

FR-10 Display a formatted report card.

---

5. Non-Functional Requirements

Performance

The application should calculate results instantly for the supported number of students.

Usability

The console interface should be simple and easy for beginners to use.

Reliability

The application should produce consistent and accurate results for valid inputs.

Maintainability

The program should be organized into reusable methods to simplify future modifications.

Input Validation

The application should reject invalid marks and prompt the user to enter correct values.

---

6. Assumptions and Constraints

Assumptions

- Marks range from **0 to 100**.
- Passing mark for each subject is **35**.
- Maximum students supported: **100**.
- Maximum subjects per student: **10**.

Constraints

- Java Console Application
- Java JDK 21 or later
- No database
- Data stored only in memory during execution
- Single-user application

---

7. Java Skills Required

Java Concept	Why It Is Needed in This Project
Variables	Store marks, totals, percentage, grades, and student details.
Data Types	Represent integers, decimal values, strings, and logical values.
Scanner Class	Read input from the keyboard.
Operators	Perform arithmetic and comparison operations.
if-else Statements	Determine grades and pass/fail status.
Nested if	Apply multiple grading conditions.
Loops	Input and process marks for multiple subjects and students.
Arrays	Store subject marks efficiently.
Methods	Divide the program into reusable modules such as calculating percentage and grade.
Modular Programming	Improves readability, maintenance, and code organization.
Basic OOP (Optional)	Represent each student as an object for better program design.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Understand real-world problem analysis.

- Design simple console-based applications.
- Work confidently with variables and data types.
- Accept and validate user input.
- Use arrays to manage collections of data.
- Apply conditional statements for decision making.
- Use loops for repetitive processing.
- Write reusable methods.
- Perform arithmetic calculations programmatically.
- Organize code using modular programming techniques.
- Develop logical thinking and problem-solving skills.
- Gain confidence in building beginner-level Java applications.

Conclusion

The **Student Grade Calculator** is an excellent beginner-level Java project that combines essential programming concepts with a practical academic application. By implementing this project, students learn how to collect input, process data, make decisions using conditional statements, iterate through data using loops, organize logic into methods, and produce meaningful output. The project lays a strong foundation for developing more advanced Java applications such as Student Management Systems, Result Processing Systems, and School Management Software.

Software Requirements Specification

Phase 1: Project 2

Bank Account Simulator

Project Type: Java Console Application
Difficulty Level: ⭑ ⭑ Intermediate Beginner
Programming Language: Java

---

1. Project Overview

1.1 Introduction

The **Bank Account Simulator** is a Java console-based application that simulates basic banking operations such as creating a bank account, depositing money, withdrawing money, checking the current balance, and viewing a mini statement of recent transactions. The project is designed to introduce learners to Object-Oriented Programming (OOP) concepts while solving a real-world banking problem.

Unlike a simple calculator program, this project models a real-world object (a bank account) and demonstrates how data and behavior can be combined using Java classes and objects.

1.2 Purpose

The purpose of this project is to help students understand how banking operations can be implemented programmatically while learning the fundamentals of Object-Oriented Programming in Java.

The application emphasizes secure handling of account information, transaction processing, and proper organization of code using classes and encapsulation.

1.3 Objectives

The objectives of this project are to:

- Simulate a simple bank account.
- Allow users to deposit money.
- Allow users to withdraw money.
- Display the current account balance.
- Maintain a mini statement of transactions.
- Demonstrate Object-Oriented Programming principles.
- Improve logical thinking and program design skills.

1.4 Scope

The project is limited to a console-based banking simulation and does not communicate with any real banking system.

The application focuses on:

- Single bank account management
- Deposit transactions

- Withdrawal transactions
- Balance inquiry
- Mini statement generation
- Input validation

1.5 Real-World Applications

The concepts learned in this project are widely used in:

- Internet Banking Systems
- ATM Software
- Mobile Banking Applications
- Digital Wallets
- Financial Management Software
- Banking Management Systems

---

2. Problem Statement

Managing bank transactions manually can lead to mistakes in calculating balances and recording transactions. Customers expect quick, accurate, and secure processing of deposits and withdrawals.

The **Bank Account Simulator** automates these banking operations by allowing users to perform transactions through a simple console application. The system maintains the account balance, validates every transaction, prevents invalid operations such as overdrawing funds, and provides a summary of account activity through a mini statement.

Expected Users

- Java beginners
- Students learning OOP
- Programming instructors
- Individuals practicing Java projects

---

3. Features

3.1 Account Creation

The system allows the user to create a bank account by entering:

- Account Number
- Account Holder Name
- Initial Balance

3.2 Deposit Money

Users can deposit money into the account.

The system:

- Validates the deposit amount.
- Updates the account balance.
- Records the transaction.

3.3 Withdraw Money

Users can withdraw money from the account.

The system:

- Checks available balance.
- Prevents withdrawal if funds are insufficient.
- Updates balance after successful withdrawal.
- Records the transaction.

3.4 Balance Inquiry

Users can view the current available account balance at any time.

3.5 Mini Statement

The application displays a summary of recent account transactions including:

- Deposits

- Withdrawals
- Current Balance

---

4. Functional Requirements

The system shall:

FR-01 Create a bank account.

FR-02 Accept deposit transactions.

FR-03 Accept withdrawal transactions.

FR-04 Prevent withdrawal beyond available balance.

FR-05 Display current account balance.

FR-06 Record each transaction.

FR-07 Display a mini statement.

FR-08 Validate all user inputs.

---

5. Non-Functional Requirements

Performance

Transactions should be processed immediately without noticeable delay.

Usability

The application should provide a simple menu-driven interface that is easy to understand.

Reliability

Every transaction should correctly update the account balance.

Maintainability

The program should be organized into separate classes and methods for easy maintenance.

Security

Sensitive account data should be protected using encapsulation.

Input Validation

The system should reject invalid amounts and display meaningful error messages.

---

6. Assumptions and Constraints

Assumptions

- Deposit amount must be greater than zero.
- Withdrawal amount must be greater than zero.
- Initial balance cannot be negative.
- Only one account is managed during program execution.
- Currency values are stored as decimal numbers.

Constraints

- Java Console Application
- Java JDK 21 or later
- No database connectivity
- Data stored only during execution
- Single-user application

---

7. Java Skills Required

Java Concept	Why It Is Needed in This Project
Classes	Represent real-world entities such as a Bank Account.
Objects	Create individual bank account instances.
Encapsulation	Protect account details by keeping data private and providing controlled access through methods.
Variables	Store account information and balances.
Data Types	Store account numbers, names, balances, and transaction amounts.
Scanner Class	Read user input from the console.
Methods	Perform banking operations such as deposit, withdrawal, and balance inquiry.
Conditional Statements	Validate transactions and prevent invalid operations.
Loops	Keep the banking menu active until the user exits.
Arrays or ArrayList (Optional)	Store transaction history for the mini statement.
Modular Programming	Divide the application into reusable components for better readability and maintenance.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Understand the fundamentals of Object-Oriented Programming.
- Design programs using classes and objects.
- Apply encapsulation to protect data.
- Implement menu-driven console applications.
- Process financial transactions logically.
- Validate user input effectively.
- Organize code into reusable methods.
- Build real-world object models.
- Strengthen problem-solving and program design skills.

Conclusion

The **Bank Account Simulator** is an excellent beginner-to-intermediate Java project that introduces learners to Object-Oriented Programming through a practical banking scenario. By implementing features such as deposits, withdrawals, balance inquiries, and transaction history,

students gain hands-on experience with classes, objects, encapsulation, methods, loops, and conditional logic. This project serves as a strong foundation for more advanced applications such as ATM Systems, Banking Management Systems, and Internet Banking Software.

Mini Software Requirements Specification (Mini SRS)

Phase 1: Project 3

Library Management System (Console)

Project Type: Java Console Application
Difficulty Level: ⭐⭐ Intermediate Beginner
Programming Language: Java

---

1. Project Overview

1.1 Introduction

The **Library Management System (Console)** is a Java console-based application designed to manage books in a small library. The system allows users to add new books, search for books, borrow available books, and return borrowed books. It simulates the basic operations of a library while introducing learners to object-oriented programming and Java's `ArrayList` collection.

Unlike projects that manage only primitive data, this project teaches students how to work with collections of objects, making it an important step toward building larger Java applications.

1.2 Purpose

The purpose of this project is to help students understand how to organize and manage multiple objects using Java collections while applying Object-Oriented Programming principles to solve a real-world problem.

1.3 Objectives

The objectives of this project are to:

- Manage a collection of books.
- Add new books to the library.
- Search books using different criteria.
- Borrow available books.
- Return borrowed books.
- Maintain the availability status of each book.
- Practice using `ArrayList` and Java objects.
- Improve logical thinking and software design skills.

1.4 Scope

The application is limited to a console-based environment and stores information only while the program is running.

The system includes:

- Book management
- Book searching
- Borrowing books
- Returning books
- Availability tracking

This project does not include user authentication, fines, due dates, or database storage.

1.5 Real-World Applications

The concepts learned in this project are used in:

- School Library Systems
- College Library Management Software
- Digital Book Catalogs
- Inventory Management Systems
- Rental Management Applications
- Book Store Inventory Software

---

2. Problem Statement

Managing library books manually becomes difficult as the number of books increases. Searching for books, tracking borrowed books, and updating their availability can be time-consuming and error-prone.

The **Library Management System** automates these operations by maintaining a digital collection of books. Users can quickly add books, search for specific titles, borrow available books, and return borrowed books while ensuring accurate availability tracking.

Expected Users

- Java beginners
- Students learning Object-Oriented Programming
- Programming instructors
- Educational institutions

---

3. Features

3.1 Add Books

Users can add new books by entering details such as:

- Book ID
- Book Title
- Author Name
- Category (optional)

Each new book is stored as an object inside an `ArrayList`.

3.2 Search Books

Users can search books using:

- Book ID
- Book Title
- Author Name

The system displays matching book information if found.

3.3 Borrow Books

Users can borrow a book if it is available.

The system:

- Checks availability.
- Marks the book as borrowed.
- Prevents borrowing an already borrowed book.

3.4 Return Books

Users can return previously borrowed books.

The system:

- Updates the availability status.
- Marks the book as available again.

3.5 Book Information Display

The application displays details such as:

- Book ID
- Title
- Author
- Availability Status

---

4. Functional Requirements

The system shall:

FR-01 Add new books to the library.

FR-02 Store books using an `ArrayList`.

FR-03 Search books by ID, title, or author.

FR-04 Allow users to borrow available books.

FR-05 Prevent borrowing unavailable books.

FR-06 Allow users to return borrowed books.

FR-07 Update book availability automatically.

FR-08 Display book information.

FR-09 Validate user input.

---

5. Non-Functional Requirements

Performance

The application should quickly perform search and update operations for a moderate number of books.

Usability

The console interface should be menu-driven, simple, and easy to navigate.

Reliability

Book availability should always remain accurate after borrow and return operations.

Maintainability

The program should be modular and organized into separate classes and methods for future enhancements.

Input Validation

The system should reject invalid or incomplete book information and display meaningful error messages.

---

6. Assumptions and Constraints

Assumptions

- Every book has a unique Book ID.
- A book can only be borrowed if it is available.
- Borrowed books must be returned before they can be borrowed again.
- The library stores a reasonable number of books in memory.

Constraints

- Java Console Application
- Java JDK 21 or later
- No database
- Data stored only during program execution
- Single-user application

---

7. Java Skills Required

Java Concept	Why It Is Needed in This Project
Classes	Represent each book as a real-world object.
Objects	Create individual book records.
ArrayList	Dynamically store and manage multiple book objects.
Variables	Store book details and availability status.
Data Types	Represent IDs, titles, authors, and status values.
Scanner Class	Read user input from the console.
Methods	Implement operations such as add, search, borrow, and return.
Conditional Statements	Check book availability and validate operations.
Loops	Traverse the `ArrayList` to search and display books.

Java Concept	Why It Is Needed in This Project
Encapsulation (Optional)	Protect book data using private fields and public methods.
Modular Programming	Organize the application into reusable and maintainable components.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Understand how real-world entities are represented as Java objects.
- Work confidently with `ArrayList`.
- Manage collections of objects dynamically.
- Search and manipulate data efficiently.
- Implement object-based program design.
- Apply conditional logic to real-world scenarios.
- Develop menu-driven applications.
- Improve problem-solving and software design skills.
- Build a foundation for larger management systems.

Conclusion

The **Library Management System (Console)** is an excellent beginner-to-intermediate Java project that introduces students to managing collections of objects using `ArrayList`. Through features such as adding books, searching records, borrowing, and returning books, learners gain practical experience with object-oriented programming, dynamic data storage, loops, methods, and decision-making. This project provides a strong foundation for developing advanced systems such as Library Information Systems, Inventory Management Software, and Database-Driven Management Applications.

Mini Software Requirements Specification

Phase 1: Project 4

Student Management System

Project Type: Java Console Application
Difficulty Level: ⭑ ⭑ ⭑ Intermediate
Programming Language: Java

1. Project Overview

1.1 Introduction

The **Student Management System** is a Java console-based application developed to manage student records efficiently. The system allows users to perform **CRUD (Create, Read, Update, Delete)** operations, search for students, and sort student records based on different criteria. It serves as a practical implementation of Object-Oriented Programming (OOP), Java Collections, and modular programming techniques.

This project is a significant step forward from basic Java applications because it involves managing multiple objects, organizing data using collections, and implementing real-world data management operations.

1.2 Purpose

The purpose of this project is to help students understand how software applications manage structured data using Java Collections and Object-Oriented Programming. It also introduces learners to CRUD operations, which form the foundation of most enterprise software applications.

1.3 Objectives

The objectives of this project are to:

- Maintain student records efficiently.
- Perform Create, Read, Update, and Delete (CRUD) operations.
- Search student records using different criteria.
- Sort student records in ascending or descending order.
- Organize code using classes and methods.
- Strengthen Object-Oriented Programming skills.
- Build a reusable and maintainable application.

1.4 Scope

The application is limited to a console-based environment and stores student data only during program execution.

The system includes:

- Student registration
- Student record management
- Record updating
- Record deletion
- Student search
- Student sorting
- Student information display

Database connectivity, authentication, and persistent storage are outside the scope of this project.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- College Student Information Systems
- School Administration Software
- University Management Systems
- Employee Management Systems
- Hospital Patient Management Systems
- Customer Management Applications (CRM)

---

2. Problem Statement

Educational institutions maintain large amounts of student information, including personal details, academic information, and identification numbers. Managing these records manually becomes difficult as the number of students grows.

The **Student Management System** automates student record management by providing facilities to add, update, search, delete, and organize student information. This reduces manual effort, minimizes errors, and improves data organization.

Expected Users

- Java students
- Programming instructors
- Educational institutions

- Beginners learning CRUD applications

---

3. Features

3.1 Add Student (Create)

The system allows users to add new student records.

Information may include:

- Student ID
- Student Name
- Age
- Course
- Department
- Marks (optional)

Each student is stored as an object in a collection.

3.2 View Students (Read)

Users can display all stored student records in a structured format.

3.3 Update Student (Update)

Users can modify existing student information using the Student ID.

Examples:

- Update name
- Update age
- Update department
- Update marks

3.4 Delete Student (Delete)

Users can remove a student record permanently using the Student ID.

3.5 Search Student

The system allows searching students using:

- Student ID
- Student Name

Matching records are displayed immediately.

3.6 Sort Student Records

The application allows sorting student records based on:

- Student ID
- Student Name
- Marks (optional)

Sorting can be performed in ascending or descending order.

3.7 Display Student Details

The application displays complete student information including:

- Student ID
- Name
- Age
- Department
- Course
- Marks (if applicable)

---

4. Functional Requirements

The system shall:

FR-01 Add new student records.

FR-02 Display all student records.

FR-03 Update existing student information.

FR-04 Delete student records.

FR-05 Search students by ID or Name.

FR-06 Sort student records.

FR-07 Store records using Java Collections.

FR-08 Validate user input.

FR-09 Prevent duplicate Student IDs.

---

5. Non-Functional Requirements

Performance

The system should efficiently manage and retrieve student records for a moderate number of students.

Usability

The application should provide a simple, menu-driven interface that is easy to use.

Reliability

Student information should remain accurate during all CRUD operations.

Maintainability

The application should be modular and organized into reusable classes and methods.

Input Validation

The system should validate all user input and reject invalid or incomplete information.

---

6. Assumptions and Constraints

Assumptions

- Every student has a unique Student ID.
- Student names cannot be empty.
- Student age must be a positive number.
- Student records are stored in memory during execution.

Constraints

- Java Console Application
- Java JDK 21 or later
- No database
- No user authentication
- Single-user application

---

7. Java Skills Required

Java Concept	Why It Is Needed in This Project
Classes	Represent each student as an object.
Objects	Create and manage student records.
Collections (`ArrayList`)	Store and manage multiple student objects dynamically.
Methods	Separate CRUD operations into reusable functions.
Object-Oriented Programming	Model real-world student entities and their behaviors.
Variables	Store student attributes such as ID, name, and age.
Scanner Class	Accept user input.

Java Concept	Why It Is Needed in This Project
Loops	Traverse collections for searching and displaying records.
Conditional Statements	Validate input and determine program flow.
Sorting Techniques	Arrange student records based on different attributes.
Encapsulation	Protect student data using private fields and getter/setter methods.
Modular Programming	Improve code readability, organization, and maintenance.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Build complete CRUD-based applications.
- Design real-world software using Object-Oriented Programming.
- Work effectively with Java Collections.
- Implement searching and sorting algorithms.
- Organize applications into multiple classes and methods.
- Validate user input and handle common errors.
- Develop reusable and maintainable Java programs.
- Strengthen logical thinking and software design skills.
- Understand the structure of enterprise management applications.

Conclusion

The **Student Management System** is an intermediate-level Java project that combines Object-Oriented Programming, Java Collections, modular programming, and CRUD operations into a practical application. By implementing student registration, searching, updating, deleting, and sorting features, learners gain valuable experience in designing structured software solutions. This project provides a strong foundation for advanced applications such as College Management Systems, University Portals, Human Resource Management Systems, and Database-Driven Enterprise Applications.

Software Requirements Specification

Phase 1: Project 5

Hospital Appointment Console System

Project Type: Java Console Application
Difficulty Level: ⭑ ⭑ ⭑ Intermediate
Programming Language: Java

---

1. Project Overview

1.1 Introduction

The **Hospital Appointment Console System** is a Java console-based application developed to automate the process of managing doctors, patients, and appointment bookings. The system allows users to register doctors and patients, schedule appointments, cancel existing appointments, and view appointment details through a menu-driven interface.

This project introduces learners to building applications involving multiple interacting objects while strengthening their understanding of **Object-Oriented Programming (OOP)**, **Java Collections (Lists)**, and **Exception Handling**.

1.2 Purpose

The purpose of this project is to simulate a basic hospital appointment management system that replaces manual appointment scheduling with a simple digital solution. It helps students learn how real-world systems manage relationships between different entities while writing structured and reliable Java programs.

1.3 Objectives

The objectives of this project are to:

- Manage doctor information.
- Manage patient information.
- Book appointments between doctors and patients.
- Cancel booked appointments.
- Prevent invalid or duplicate appointments.
- Handle unexpected user input using exception handling.
- Practice Object-Oriented Programming and Java Collections.

1.4 Scope

The application is limited to a console-based environment and stores all information only during program execution.

The system includes:

- Doctor management
- Patient management
- Appointment booking
- Appointment cancellation
- Appointment viewing
- Input validation
- Exception handling

The project does not include user authentication, medical records, billing, prescriptions, or database integration.

1.5 Real-World Applications

The concepts implemented in this project are commonly used in:

- Hospital Management Systems
- Clinic Appointment Software
- Online Doctor Booking Platforms
- Healthcare Information Systems
- Medical Scheduling Applications
- Patient Registration Systems

---

2. Problem Statement

Hospitals and clinics often manage appointments between doctors and patients. When appointments are handled manually, double bookings, scheduling conflicts, and record management errors can occur. Manual systems also consume more time and reduce operational efficiency.

The **Hospital Appointment Console System** automates appointment scheduling by maintaining doctor and patient information, allowing appointment booking and cancellation while ensuring that data is organized and operations are performed correctly.

Expected Users

- Java learners
- Programming instructors
- Educational institutions
- Students practicing Object-Oriented Programming
- Beginners developing management system projects

---

3. Features

3.1 Doctor Management

The system allows users to register doctors by entering:

- Doctor ID
- Doctor Name
- Specialization

Doctors are stored as objects in a list.

3.2 Patient Management

The system allows users to register patients by entering:

- Patient ID
- Patient Name
- Age
- Contact Number

Patient records are stored in a list.

3.3 Appointment Booking

Users can book appointments by selecting:

- Patient
- Doctor
- Appointment Date
- Appointment Time

The system verifies that the selected doctor is available before confirming the booking.

3.4 Cancel Appointment

Users can cancel an existing appointment.

The system updates the appointment records accordingly.

3.5 View Appointments

Users can display all scheduled appointments, including:

- Appointment ID
- Doctor Name
- Patient Name
- Date
- Time
- Appointment Status

3.6 Exception Handling

The application handles common runtime errors, including:

- Invalid numeric input
- Empty fields
- Booking unavailable doctors
- Selecting non-existent doctors or patients
- Invalid menu choices

---

4. Functional Requirements

The system shall:

FR-01 Register doctors.

FR-02 Register patients.

FR-03 Store doctors and patients using Java Lists.

FR-04 Book appointments.

FR-05 Prevent duplicate appointment bookings for the same doctor at the same date and time.

FR-06 Cancel appointments.

FR-07 Display appointment details.

FR-08 Validate all user inputs.

FR-09 Handle invalid operations using exception handling.

---

5. Non-Functional Requirements

Performance

The application should process appointment requests and searches quickly for a moderate number of records.

Usability

The system should provide a simple and user-friendly menu-driven console interface.

Reliability

Appointment information should remain consistent during booking and cancellation operations.

Maintainability

The application should be organized into reusable classes and methods to simplify future enhancements.

Input Validation

The system should reject invalid input and provide meaningful error messages instead of terminating unexpectedly.

---

6. Assumptions and Constraints

Assumptions

- Every doctor has a unique Doctor ID.
- Every patient has a unique Patient ID.
- One doctor cannot have two appointments at the same date and time.
- Appointment IDs are unique.
- Appointment dates and times are entered in a valid format.

Constraints

- Java Console Application
- Java JDK 21 or later
- No database
- Data stored only during program execution
- Single-user application

---

7. Java Skills Required

Java Concept	Why It Is Needed in This Project
Classes	Represent doctors, patients, and appointments as separate objects.
Objects	Create real-world entities and establish relationships between them.

Java Concept	Why It Is Needed in This Project
Object-Oriented Programming	Model hospital operations using multiple interacting classes.
Lists (`ArrayList`)	Store collections of doctors, patients, and appointments dynamically.
Methods	Separate functionalities such as booking, cancellation, registration, and searching.
Variables	Store IDs, names, dates, times, and other details.
Scanner Class	Accept user input from the console.
Conditional Statements	Validate bookings and prevent scheduling conflicts.
Loops	Search and display records from collections.
Exception Handling (`try-catch`)	Handle invalid user input and runtime errors without crashing the application.
Encapsulation	Protect object data using private fields with getter and setter methods.
Modular Programming	Improve code organization, readability, and maintainability.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Design applications involving multiple interacting classes.
- Build real-world management systems using Object-Oriented Programming.
- Work efficiently with Java Lists.
- Handle invalid input using exception handling.
- Develop reliable menu-driven applications.
- Implement booking and cancellation workflows.
- Validate user input effectively.
- Organize software into reusable methods and classes.
- Strengthen problem-solving and software design skills.

Conclusion

The **Hospital Appointment Console System** is an intermediate-level Java project that introduces learners to developing real-world management software using Object-Oriented Programming, Java Collections, and Exception Handling. By implementing doctor registration, patient management, appointment booking, cancellation, and input validation, students gain practical experience in designing structured, modular, and reliable applications. This project

provides an excellent foundation for advanced healthcare systems, enterprise management applications, and database-driven software development.

Software Requirements Specification

Phase 2: Project 6

Expense Tracker

Project Type: Java Console Application
Difficulty Level: ⭑ ⭑ ⭑ Intermediate
Programming Language: Java

---

1. Project Overview

1.1 Introduction

The **Expense Tracker** is a Java console-based application designed to help users record, organize, manage, and analyze their daily financial expenses. The application enables users to add, update, delete, search, and categorize expenses while maintaining a history of transactions using file storage.

This project introduces learners to intermediate Java programming concepts such as **Collections**, **File Handling**, **Exception Handling**, **Generics**, **Comparable**, **Comparator**, and **Lambda Expressions** by solving a practical personal finance problem.

1.2 Purpose

The purpose of this project is to develop a simple personal finance management application that allows users to monitor their spending habits and maintain expense records without relying on manual calculations.

The project also strengthens understanding of Java collections, persistent storage using files, and modern Java programming techniques.

1.3 Objectives

The objectives of this project are to:

- Record daily expenses.
- Organize expenses into categories.
- Search expense records.
- Update existing expenses.
- Delete unwanted expense records.
- Calculate total spending.
- Generate category-wise expense summaries.
- Save and load data using files.
- Practice intermediate Java programming concepts.

1.4 Scope

The application manages personal expense records through a console-based interface.

The system includes:

- Expense management
- Expense categorization
- Search functionality
- Expense editing
- Expense deletion
- Expense summary
- File-based data storage
- Sorting expense records

The application does not include online banking integration or cloud synchronization.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- Personal Finance Applications
- Budget Planning Software
- Expense Management Systems
- Accounting Software

- Financial Tracking Applications
- Mobile Budget Tracker Apps

---

2. Problem Statement

Many people record their daily expenses manually using notebooks or spreadsheets. Over time, managing these records becomes difficult, making it hard to identify spending patterns and control unnecessary expenses.

The **Expense Tracker** automates expense recording and organization by storing financial transactions digitally. Users can easily manage expenses, search previous records, calculate total expenditures, and generate useful spending summaries.

Expected Users

- Students
- Individual users
- Java learners
- Programming instructors
- Anyone learning personal finance application development

---

3. Features

3.1 Add Expense

Users can record a new expense by entering:

- Expense ID
- Expense Title
- Category
- Amount
- Date

3.2 View Expenses

Display all stored expense records in a structured format.

3.3 Update Expense

Users can modify existing expense details using the Expense ID.

3.4 Delete Expense

Remove unwanted expense records from the system.

3.5 Search Expenses

Search expenses using:

- Expense ID
- Category
- Date
- Title

3.6 Expense Summary

Display:

- Total Expenses
- Category-wise Expenses
- Number of Transactions
- Highest Expense
- Lowest Expense

3.7 Sorting

Sort expenses based on:

- Date
- Amount
- Category

using Comparable or Comparator.

3.8 File Storage

Save all expense records into a file and automatically reload them when the application starts.

---

4. Functional Requirements

The system shall:

FR-01 Add expense records.

FR-02 View all expenses.

FR-03 Update expense information.

FR-04 Delete expense records.

FR-05 Search expenses.

FR-06 Generate expense summaries.

FR-07 Sort expenses.

FR-08 Save expense data to files.

FR-09 Load expense data from files.

FR-10 Handle invalid user input using exception handling.

---

5. Non-Functional Requirements

Performance

The application should quickly process expense records and summaries.

Usability

The system should provide a simple menu-driven interface suitable for beginners.

Reliability

Expense records should remain accurate after every operation.

Maintainability

The code should be modular and easy to extend.

Security

The application should validate all user inputs before processing.

Persistence

Expense records should remain available after restarting the application through file storage.

---

6. Assumptions and Constraints

Assumptions

- Every expense has a unique Expense ID.
- Expense amount must be greater than zero.
- Categories are predefined or user-defined.
- Data is stored in local files.

Constraints

- Java Console Application
- Java JDK 21 or later
- No database
- Local file storage only
- Single-user application

---

7. Java Skills Required

Java Concept	Why It Is Needed in This Project
Classes & Objects	Represent each expense as an object.
Collections (`ArrayList`)	Store multiple expense records dynamically.
Generics	Ensure type-safe collections.
File Handling	Save and retrieve expense records from files.
Exception Handling	Prevent program crashes due to invalid input or file errors.
Comparable	Sort expenses naturally (e.g., by date or ID).
Comparator	Sort expenses using different criteria such as amount or category.
Lambda Expressions	Simplify sorting and filtering logic using Java functional programming.
Methods	Separate each operation into reusable modules.
Loops	Traverse expense collections.
Conditional Statements	Validate data and perform logical decisions.
Scanner Class	Read user input.
Modular Programming	Improve maintainability and readability.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Build a complete CRUD-based Java application.
- Store application data permanently using files.
- Work confidently with Java Collections.
- Use Generics for type-safe programming.
- Implement file reading and writing operations.
- Handle exceptions gracefully.
- Sort objects using Comparable and Comparator.
- Use Lambda Expressions to write cleaner code.
- Design modular and maintainable Java applications.
- Develop practical personal finance software.

Conclusion

The **Expense Tracker** is an intermediate-level Java project that introduces persistent data storage, advanced collection handling, and modern Java programming techniques. Through expense management, searching, sorting, reporting, and file handling, learners gain practical experience in building real-world desktop applications. This project serves as an excellent foundation for larger financial management systems, accounting software, and enterprise Java applications.

Software Requirements Specification (SRS)

Phase 2: Project 7

Inventory Management System

Project Type: Java Console Application
Difficulty Level: ⭑ ⭑ ⭑ Intermediate
Programming Language: Java

---

1. Project Overview

1.1 Introduction

The **Inventory Management System** is a Java console-based application developed to efficiently manage the inventory of a store or organization. It enables users to add new products, update existing product information, search products, delete products, manage stock quantities, and generate inventory reports.

The application demonstrates how inventory records can be maintained digitally using Java Collections, File Handling, Exception Handling, and Object-Oriented Programming. It provides learners with practical experience in developing business-oriented software solutions.

1.2 Purpose

The purpose of this project is to replace manual inventory tracking with a digital inventory management solution. The system allows users to organize product information, monitor stock levels, reduce human errors, and simplify inventory operations while strengthening intermediate Java programming skills.

1.3 Objectives

The objectives of this project are to:

- Maintain product records efficiently.
- Manage product stock levels.
- Perform CRUD (Create, Read, Update, Delete) operations.
- Search products quickly.
- Sort inventory records.
- Save inventory information permanently using files.
- Handle runtime errors gracefully using exception handling.
- Practice advanced Java programming concepts.

1.4 Scope

The Inventory Management System manages product information within a console-based environment.

The system supports:

- Product registration
- Product search
- Product modification
- Product deletion
- Stock management
- Inventory reporting
- File-based data storage
- Sorting inventory records

This project does not include barcode scanning, supplier management, online purchasing, or database connectivity.

1.5 Real-World Applications

The concepts implemented in this project are commonly used in:

- Retail Store Management Systems
- Warehouse Management Systems
- Supermarket Inventory Systems
- Pharmacy Inventory Software
- Bookstore Management Systems
- Manufacturing Inventory Solutions
- E-commerce Product Management Systems

---

2. Problem Statement

Businesses maintain thousands of products with varying stock quantities. Managing inventory manually often leads to incorrect stock information, duplicate records, misplaced products, and delayed updates.

The **Inventory Management System** automates inventory management by maintaining digital product records. It allows users to update stock quantities, search products instantly, generate inventory reports, and maintain accurate inventory information.

Expected Users

- Store Managers
- Shop Owners
- Warehouse Staff
- Students learning Java
- Programming instructors

---

3. Features

3.1 Product Registration

Users can add new products by entering:

- Product ID
- Product Name
- Category

- Price
- Quantity in Stock

Each product is stored as an object.

3.2 View Inventory

Display all available products in a well-formatted table.

Displayed information includes:

- Product ID
- Product Name
- Category
- Price
- Quantity
- Stock Status

3.3 Search Products

Users can search products using:

- Product ID
- Product Name
- Category

Matching products are displayed immediately.

3.4 Update Product Information

Users can modify:

- Product Name
- Category
- Price
- Stock Quantity

using the Product ID.

3.5 Delete Product

Users can permanently remove products from the inventory.

3.6 Stock Management

The application automatically updates inventory quantities whenever stock is added or removed.

It also identifies:

- In Stock
- Low Stock
- Out of Stock

products.

3.7 Inventory Reports

Generate reports showing:

- Total Products
- Total Stock Quantity
- Available Products
- Low Stock Products
- Out of Stock Products

3.8 Sorting

Sort inventory based on:

- Product Name
- Product ID
- Price
- Quantity

using Comparable or Comparator.

3.9 File Storage

Save inventory records into files and automatically restore them when the application starts.

---

4. Functional Requirements

The system shall:

FR-01 Add new product records.

FR-02 Display all inventory items.

FR-03 Search products using multiple criteria.

FR-04 Update existing product information.

FR-05 Delete products.

FR-06 Manage stock quantities.

FR-07 Generate inventory reports.

FR-08 Sort inventory records.

FR-09 Save inventory data to files.

FR-10 Load inventory data from files.

FR-11 Validate all user inputs.

FR-12 Handle exceptions without terminating the application.

---

5. Non-Functional Requirements

Performance

The system should efficiently manage hundreds of inventory records with minimal response time.

Reliability

Inventory data should remain accurate after every CRUD operation.

Usability

The application should provide a simple menu-driven interface that is easy to understand.

Maintainability

The application should use modular programming practices to simplify future maintenance.

Scalability

The design should allow additional inventory features to be added without major code modifications.

Data Persistence

Inventory information should remain available after application restart through local file storage.

Input Validation

All invalid user input should be detected and handled appropriately.

---

6. Assumptions and Constraints

Assumptions

- Every product has a unique Product ID.
- Product price must be greater than zero.
- Stock quantity cannot be negative.
- Product names cannot be empty.
- Data is stored in local files.

Constraints

- Java Console Application
- Java JDK 21 or later
- No relational database
- Local file storage only
- Single-user application

---

7. Java Skills Required

Java Concept	Application in the Project
Classes & Objects	Represent each product as an object containing inventory information.
Object-Oriented Programming	Model products and inventory operations using classes and objects.
Collections (`ArrayList`)	Dynamically store and manage multiple product objects.
Generics	Ensure type-safe collection management.
File Handling	Save and retrieve inventory records from files.
Exception Handling	Handle invalid user input and file-related errors safely.
Comparable	Sort products naturally, such as by Product ID or Name.
Comparator	Sort products by price, quantity, or category.
Lambda Expressions	Simplify sorting and searching operations using functional programming.
Methods	Organize inventory operations into reusable modules.
Loops	Traverse inventory collections efficiently.
Conditional Statements	Validate stock information and perform business logic.

Java Concept	Application in the Project
Scanner Class	Accept user input through the console.
Modular Programming	Improve code readability, testing, and maintainability.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop complete inventory management applications.
- Apply CRUD operations in real-world scenarios.
- Design business applications using Object-Oriented Programming.
- Store and retrieve data using Java File Handling.
- Work efficiently with Java Collections and Generics.
- Sort objects using Comparable and Comparator interfaces.
- Use Lambda Expressions to simplify collection processing.
- Handle runtime exceptions professionally.
- Build scalable and maintainable Java applications.
- Understand the software architecture of inventory-based enterprise systems.

Conclusion

The **Inventory Management System** is an intermediate-level Java project that integrates Object-Oriented Programming, Collections, File Handling, Exception Handling, Generics, Comparable, Comparator, and Lambda Expressions into a practical business application. By implementing product management, stock tracking, inventory reporting, and persistent storage, learners gain hands-on experience in designing structured and maintainable software. This project provides a strong foundation for developing enterprise applications such as Warehouse Management Systems, Retail Management Software, Supply Chain Solutions, and ERP systems.

Software Requirements Specification (SRS)

Phase 2: Project 8

Quiz Application

Project Type: Java Console Application
Difficulty Level: ⭐⭐⭐ Intermediate
Programming Language: Java

---

1. Project Overview

1.1 Introduction

The **Quiz Application** is a Java console-based application designed to conduct multiple-choice quizzes in an interactive environment. The system allows administrators to manage quiz questions and enables users to attempt quizzes, receive scores, and view their performance immediately after completion.

The project introduces learners to intermediate Java concepts such as **Collections**, **File Handling**, **Exception Handling**, **Generics**, **Comparable**, **Comparator**, and **Lambda Expressions** while demonstrating how educational assessment systems are built.

1.2 Purpose

The purpose of this project is to develop an automated quiz system that replaces paper-based or manually evaluated quizzes. The application provides instant score calculation, accurate answer evaluation, and persistent storage of quiz questions using files.

1.3 Objectives

The objectives of this project are to:

- Create and manage quiz questions.
- Conduct multiple-choice quizzes.
- Evaluate user answers automatically.
- Calculate quiz scores instantly.
- Display quiz results and performance.
- Save quiz data using file handling.
- Practice intermediate Java programming concepts.

1.4 Scope

The application is limited to a console-based environment and stores quiz questions and results using local files.

The system includes:

- Question management
- Quiz participation
- Automatic evaluation
- Score calculation
- Result display
- Question searching
- Question sorting
- File-based storage

This project does not include online multiplayer quizzes, networking, user authentication, or database integration.

1.5 Real-World Applications

The concepts implemented in this project are used in:

- Online Examination Systems
- Learning Management Systems (LMS)
- Competitive Exam Platforms
- Certification Portals
- School Assessment Systems
- Employee Training Platforms

---

2. Problem Statement

Educational institutions and organizations frequently conduct quizzes to evaluate knowledge. Manual quiz evaluation consumes time, increases the possibility of calculation errors, and delays result publication.

The **Quiz Application** automates the complete quiz process by managing questions digitally, evaluating answers instantly, calculating scores accurately, and displaying immediate feedback to users.

Expected Users

- Students
- Teachers
- Educational Institutions
- Java Learners
- Programming Instructors

---

3. Features

3.1 Question Management

Users (or administrators) can add new quiz questions by entering:

- Question ID
- Question Text
- Four Options (A, B, C, D)
- Correct Answer
- Difficulty Level (Optional)

Questions are stored as objects in a collection.

3.2 View Questions

Display all stored quiz questions with their details (excluding answers during quiz mode).

3.3 Search Questions

Users can search questions using:

- Question ID
- Difficulty Level
- Keywords

3.4 Start Quiz

Users can attempt the quiz.

The application:

- Displays questions one by one.
- Accepts user answers.
- Validates responses.
- Moves to the next question automatically.

3.5 Automatic Evaluation

After quiz completion, the system compares user answers with correct answers and calculates:

- Correct Answers
- Wrong Answers
- Total Questions
- Final Score
- Percentage

3.6 Result Display

Display:

- User Score
- Percentage
- Pass/Fail Status
- Number of Correct Answers
- Number of Incorrect Answers
- Performance Summary

3.7 Sorting

Sort questions based on:

- Question ID
- Difficulty Level
- Question Category (Optional)

using Comparable or Comparator.

3.8 File Storage

Save quiz questions and quiz results into local files and reload them automatically when the application starts.

---

4. Functional Requirements

The system shall:

FR-01 Add quiz questions.

FR-02 Display all questions.

FR-03 Search quiz questions.

FR-04 Conduct quizzes.

FR-05 Evaluate user answers automatically.

FR-06 Calculate quiz scores.

FR-07 Display quiz results.

FR-08 Sort quiz questions.

FR-09 Save quiz data using files.

FR-10 Load quiz data from files.

FR-11 Validate user input.

FR-12 Handle runtime exceptions gracefully.

---

5. Non-Functional Requirements

Performance

The system should evaluate quizzes and calculate scores instantly.

Reliability

The application should consistently produce accurate quiz results.

Usability

The system should provide an easy-to-use, menu-driven console interface.

Maintainability

The application should be modular and organized for future feature additions.

Scalability

The system should support adding a large number of quiz questions without requiring structural changes.

Data Persistence

Quiz questions and user results should be stored permanently using local files.

Input Validation

The application should validate user choices and reject invalid responses.

---

6. Assumptions and Constraints

Assumptions

- Every question has a unique Question ID.
- Each question has exactly one correct answer.
- Questions are multiple-choice.
- Quiz data is stored locally.
- Users answer one question at a time.

Constraints

- Java Console Application
- Java JDK 21 or later
- Local file storage only
- No database
- Single-user application

---

7. Java Skills Required

Java Concept	Application in the Project
Classes & Objects	Represent each quiz question and quiz result as objects.
Object-Oriented Programming	Model questions, quizzes, and results using classes.
Collections (`ArrayList`)	Store and manage quiz questions dynamically.
Generics	Ensure type-safe collections.
File Handling	Store quiz questions and quiz results permanently.
Exception Handling	Handle invalid input and file operation errors safely.
Comparable	Sort questions naturally, such as by Question ID.
Comparator	Sort questions based on difficulty level or category.
Lambda Expressions	Simplify sorting, searching, and filtering operations.
Methods	Organize quiz operations into reusable functions.
Loops	Display questions and process user responses.
Conditional Statements	Evaluate answers and determine quiz results.
Scanner Class	Read user responses from the console.
Modular Programming	Improve readability, maintainability, and code reuse.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop a complete console-based assessment application.
- Work with Collections and Generics to manage quiz data.
- Store and retrieve information using File Handling.
- Implement automatic answer evaluation algorithms.
- Apply Exception Handling to create robust applications.
- Sort objects using Comparable and Comparator.
- Use Lambda Expressions for concise collection processing.
- Design modular Object-Oriented Java applications.
- Strengthen logical thinking and software design skills.
- Understand the architecture of online examination systems.

Conclusion

The **Quiz Application** is an intermediate-level Java project that combines Object-Oriented Programming, Collections, File Handling, Exception Handling, Generics, Comparable, Comparator, and Lambda Expressions into a practical educational software solution. By implementing question management, automated evaluation, score calculation, and persistent storage, learners gain hands-on experience in building structured and maintainable applications. This project provides a solid foundation for developing advanced systems such as Online Examination Platforms, Learning Management Systems (LMS), Certification Portals, and Computer-Based Testing (CBT) applications.

Software Requirements Specification (SRS)

Phase 2: Project 9

Employee Payroll System

Project Type: Java Console Application
Difficulty Level: ⭑ ⭑ ⭑ Intermediate
Programming Language: Java

---

1. Project Overview

1.1 Introduction

The **Employee Payroll System** is a Java console-based application designed to automate employee salary management within an organization. The system enables users to maintain employee records, calculate salaries, generate payroll reports, and manage employee information efficiently.

The application introduces learners to real-world business application development by combining **Object-Oriented Programming (OOP)**, **Collections**, **File Handling**, **Exception Handling**, **Generics**, **Comparable**, **Comparator**, and **Lambda Expressions**.

1.2 Purpose

The purpose of this project is to replace manual payroll calculations with an automated system that accurately computes employee salaries, stores payroll information, and generates salary reports while strengthening intermediate Java programming skills.

1.3 Objectives

The objectives of this project are to:

- Maintain employee records.
- Calculate employee salaries automatically.
- Generate payroll reports.
- Search employee information.
- Update and delete employee records.
- Store payroll data using files.
- Practice modern Java programming concepts.

1.4 Scope

The application manages employee information through a console-based interface.

The system includes:

- Employee registration
- Salary calculation
- Payroll generation

- Employee search
- Employee update
- Employee deletion
- Salary reports
- File-based data storage

The application does not include tax filing, attendance systems, biometric integration, or database connectivity.

1.5 Real-World Applications

The concepts implemented in this project are commonly used in:

- Human Resource Management Systems (HRMS)
- Payroll Management Software
- Enterprise Resource Planning (ERP) Systems
- Corporate Employee Management Systems
- Government Payroll Systems
- Accounting Software

---

2. Problem Statement

Organizations employ multiple staff members with different salary structures. Managing payroll manually is time-consuming and increases the risk of salary calculation errors, duplicate records, and inconsistent payroll reports.

The **Employee Payroll System** automates employee management and payroll processing by maintaining employee information digitally, calculating salaries accurately, and generating organized payroll reports.

Expected Users

- Human Resource Departments
- Payroll Administrators
- Company Managers
- Java Learners
- Programming Instructors

---

3. Features

3.1 Employee Registration

Users can register employees by entering:

- Employee ID
- Employee Name
- Department
- Designation
- Basic Salary

Each employee is stored as an object.

3.2 Employee Management

Users can:

- View Employee Details
- Update Employee Information
- Delete Employee Records

3.3 Salary Calculation

The system automatically calculates:

- Basic Salary
- Allowances
- Bonuses (Optional)
- Deductions (Optional)
- Net Salary

Salary calculation follows predefined business rules.

3.4 Search Employee

Users can search employees using:

- Employee ID
- Employee Name
- Department

3.5 Payroll Report

Generate reports displaying:

- Employee Details
- Salary Breakdown
- Net Salary
- Payroll Summary

3.6 Sorting

Employee records can be sorted based on:

- Employee ID
- Employee Name
- Department
- Salary

using Comparable or Comparator.

3.7 File Storage

Employee information and payroll records are saved to local files and automatically restored when the application starts.

---

4. Functional Requirements

The system shall:

FR-01 Register employees.

FR-02 Display employee information.

FR-03 Update employee records.

FR-04 Delete employee records.

FR-05 Calculate employee salaries.

FR-06 Search employees.

FR-07 Generate payroll reports.

FR-08 Sort employee records.

FR-09 Save payroll information to files.

FR-10 Load payroll information from files.

FR-11 Validate all user inputs.

FR-12 Handle runtime exceptions gracefully.

---

5. Non-Functional Requirements

Performance

The system should generate payroll reports quickly for a moderate number of employees.

Reliability

Salary calculations should always be accurate and consistent.

Usability

The application should provide a simple menu-driven interface suitable for beginners.

Maintainability

The code should follow modular programming principles to simplify future enhancements.

Scalability

The application should support future payroll features without major architectural changes.

Data Persistence

Employee and payroll records should be permanently stored using local files.

Input Validation

The system should validate salary values, employee information, and all user inputs before processing.

---

6. Assumptions and Constraints

Assumptions

- Every employee has a unique Employee ID.
- Salary values cannot be negative.
- Employee names cannot be empty.
- Payroll calculations follow predefined salary rules.
- Data is stored locally using files.

Constraints

- Java Console Application
- Java JDK 21 or later
- No database
- Local file storage only
- Single-user application

---

7. Java Skills Required

Java Concept	Application in the Project
Classes & Objects	Represent employees and payroll records as objects.
Object-Oriented Programming	Model employee information and payroll operations.
Collections (`ArrayList`)	Store employee records dynamically.
Generics	Provide type-safe collection management.
File Handling	Save and retrieve employee and payroll information.
Exception Handling	Handle invalid salary input and file-related errors safely.
Comparable	Sort employees naturally by Employee ID or Name.
Comparator	Sort employees based on salary, department, or designation.
Lambda Expressions	Simplify sorting, searching, and filtering operations.
Methods	Organize payroll operations into reusable functions.
Loops	Traverse employee collections efficiently.
Conditional Statements	Calculate salaries and validate payroll rules.
Scanner Class	Accept user input through the console.
Modular Programming	Improve readability, maintainability, and scalability.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop business-oriented Java applications.
- Build complete CRUD-based payroll systems.
- Apply Object-Oriented Programming to enterprise scenarios.
- Use Collections and Generics effectively.
- Store business data using File Handling.
- Handle exceptions professionally.
- Implement sorting using Comparable and Comparator.
- Apply Lambda Expressions for cleaner collection processing.
- Design modular and maintainable software.
- Understand the architecture of payroll and HR management systems.

Conclusion

The **Employee Payroll System** is an intermediate-level Java project that simulates real-world payroll processing within an organization. By implementing employee management, salary calculation, payroll reporting, and persistent file storage, learners gain practical experience with Object-Oriented Programming, Collections, File Handling, Exception Handling, Generics, Comparable, Comparator, and Lambda Expressions. This project serves as an excellent foundation for developing enterprise Human Resource Management Systems (HRMS), Payroll Software, ERP solutions, and other business applications.

Software Requirements Specification (SRS)

Phase 2: Project 10

Movie Ticket Booking System

Project Type: Java Console Application
Difficulty Level: ⭑ ⭑ ⭑ Intermediate
Programming Language: Java

---

1. Project Overview

1.1 Introduction

The **Movie Ticket Booking System** is a Java console-based application designed to automate the process of booking movie tickets. The application enables users to browse available movies, view show timings, select seats, book tickets, cancel bookings, and generate booking summaries.

This project provides practical experience in developing a real-world reservation system while strengthening knowledge of **Object-Oriented Programming (OOP)**, **Collections**, **File Handling**, **Exception Handling**, **Generics**, **Comparable**, **Comparator**, and **Lambda Expressions**.

1.2 Purpose

The purpose of this project is to replace manual ticket booking with a digital reservation system that efficiently manages movies, seat availability, and customer bookings. It also helps learners understand how reservation systems are designed using Java.

1.3 Objectives

The objectives of this project are to:

- Maintain movie information.
- Manage movie show timings.
- Book movie tickets.
- Cancel existing bookings.
- Track seat availability.
- Generate booking reports.
- Store booking information using files.
- Practice intermediate Java programming concepts.

1.4 Scope

The application manages movie bookings in a console-based environment.

The system supports:

- Movie management
- Show management
- Seat booking
- Booking cancellation
- Seat availability tracking
- Booking reports
- File-based data storage

The application does not include online payment, user authentication, QR code generation, or database connectivity.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- Online Movie Ticket Booking Platforms
- Cinema Management Systems
- Event Reservation Systems
- Railway Reservation Systems
- Airline Booking Systems

- Hotel Reservation Software

---

2. Problem Statement

Traditional ticket booking methods often involve manual seat allocation and record management, increasing the possibility of double bookings, inaccurate seat availability, and inefficient customer service.

The **Movie Ticket Booking System** automates ticket reservation by maintaining digital records of movies, shows, available seats, and customer bookings. It ensures accurate seat allocation, simplifies booking and cancellation, and provides quick access to booking information.

Expected Users

- Cinema Staff
- Theatre Managers
- Java Learners
- Programming Instructors
- Students building reservation system projects

---

3. Features

3.1 Movie Management

Users can add and manage movie details such as:

- Movie ID
- Movie Name
- Genre
- Duration
- Language

Each movie is stored as an object.

3.2 Show Management

The system allows users to manage movie shows by specifying:

- Show ID
- Movie
- Show Date
- Show Time
- Available Seats

3.3 Ticket Booking

Users can book tickets by selecting:

- Movie
- Show
- Number of Seats

The system automatically updates seat availability after successful booking.

3.4 Cancel Booking

Users can cancel previously booked tickets.

The system restores the cancelled seats to the available seat count.

3.5 Seat Availability

Users can check the number of available seats before making a booking.

The system prevents bookings when sufficient seats are unavailable.

3.6 Booking Summary

Display booking information including:

- Booking ID
- Customer Name
- Movie Name
- Show Date

- Show Time
- Number of Seats
- Booking Status

3.7 Sorting

Sort records based on:

- Movie Name
- Show Time
- Available Seats
- Booking ID

using Comparable or Comparator.

3.8 File Storage

Store movie details, show information, and booking records in local files. Automatically load the saved data when the application starts.

---

4. Functional Requirements

The system shall:

FR-01 Add and manage movie records.

FR-02 Manage movie shows.

FR-03 Display available movies.

FR-04 Book movie tickets.

FR-05 Cancel ticket bookings.

FR-06 Update seat availability automatically.

FR-07 Display booking summaries.

FR-08 Sort movie and booking records.

FR-09 Save booking information using files.

FR-10 Load saved booking information from files.

FR-11 Validate all user input.

FR-12 Handle runtime exceptions gracefully.

---

5. Non-Functional Requirements

Performance

The application should process bookings and cancellations quickly with minimal response time.

Reliability

Seat availability and booking information should remain accurate after every transaction.

Usability

The application should provide a simple, menu-driven console interface suitable for beginners.

Maintainability

The code should follow modular programming principles to simplify future enhancements.

Scalability

The application should support additional movies and show schedules without significant code modifications.

Data Persistence

Movie, show, and booking information should remain available after restarting the application through local file storage.

Input Validation

The system should validate booking requests, seat counts, and user information before processing.

---

6. Assumptions and Constraints

Assumptions

- Every movie has a unique Movie ID.
- Every show has a unique Show ID.
- Every booking has a unique Booking ID.
- Available seats cannot be negative.
- Seat bookings cannot exceed available seats.
- Data is stored locally using files.

Constraints

- Java Console Application
- Java JDK 21 or later
- Local file storage only
- No relational database
- Single-user application

---

7. Java Skills Required

Java Concept	Application in the Project

Java Concept	Application in the Project
Classes & Objects	Represent movies, shows, customers, and bookings as separate objects.
Object-Oriented Programming	Model the reservation system using interacting classes.
Collections (`ArrayList`)	Dynamically store movies, shows, and booking records.
Generics	Ensure type-safe collection management.
File Handling	Save and retrieve booking and movie information from local files.
Exception Handling	Handle invalid booking requests and file operation errors safely.
Comparable	Sort records naturally by Movie ID or Booking ID.
Comparator	Sort movies by name, shows by time, or bookings by seat count.
Lambda Expressions	Simplify sorting, filtering, and searching operations.
Methods	Organize booking operations into reusable functions.
Loops	Traverse movie and booking collections efficiently.
Conditional Statements	Validate seat availability and booking conditions.
Scanner Class	Accept user input through the console.
Modular Programming	Improve readability, maintainability, and scalability.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop a complete reservation-based Java application.
- Model multiple real-world entities using Object-Oriented Programming.
- Work efficiently with Collections and Generics.
- Implement persistent storage using File Handling.
- Handle runtime exceptions professionally.
- Apply Comparable and Comparator for flexible sorting.
- Use Lambda Expressions to simplify collection processing.
- Design modular and maintainable software.
- Understand booking workflows and seat management algorithms.
- Build a strong foundation for advanced reservation and management systems.

Conclusion

The **Movie Ticket Booking System** is an intermediate-level Java project that simulates a real-world ticket reservation platform. By implementing movie management, show scheduling, seat booking, cancellation, booking summaries, and persistent storage, learners gain practical

experience with Object-Oriented Programming, Collections, File Handling, Exception Handling, Generics, Comparable, Comparator, and Lambda Expressions. This project prepares learners for developing larger reservation-based applications such as Airline Reservation Systems, Railway Booking Systems, Hotel Reservation Software, and Event Management Platforms.

Software Requirements Specification (SRS)

Phase 3: Project 11

Chat Application (Socket Programming)

Project Type: Java Client-Server Console Application
Difficulty Level: ⭑ ⭑ ⭑ ⭑ Above Intermediate
Programming Language: Java

---

1. Project Overview

1.1 Introduction

The **Chat Application** is a Java-based client-server application that enables multiple users to communicate over a network using socket programming. The system consists of a server that accepts client connections and clients that exchange messages in real time.

This project introduces learners to **Networking**, **Socket Programming**, **Multithreading**, **Object-Oriented Programming (OOP)**, **Collections**, **Exception Handling**, and modular software design while demonstrating how modern communication applications work.

1.2 Purpose

The purpose of this project is to develop a real-time communication system that allows users connected through a network to exchange text messages instantly.

The project also provides practical experience with client-server architecture, concurrent programming, and network communication.

1.3 Objectives

The objectives of this project are to:

- Establish communication using sockets.
- Build separate server and client applications.
- Support multiple clients simultaneously.
- Broadcast messages between connected users.
- Handle client connections and disconnections.
- Practice multithreading and networking concepts.
- Improve software architecture and concurrent programming skills.

1.4 Scope

The application provides a console-based messaging platform.

The system includes:

- Server initialization
- Client connection
- User registration
- Real-time messaging
- Broadcast communication
- Client disconnection
- Exception handling
- Thread management

Database storage, multimedia messaging, authentication, and encryption are outside the scope of this project.

1.5 Real-World Applications

The concepts implemented in this project are used in:

- Instant Messaging Applications
- Chat Servers
- Online Collaboration Tools
- Multiplayer Games
- Customer Support Systems
- Video Conferencing Platforms
- Distributed Systems

---

2. Problem Statement

Communication between multiple users over a network requires continuous message exchange and simultaneous handling of multiple connections. Manual communication methods or single-user programs cannot support real-time interaction between multiple users.

The **Chat Application** solves this problem by implementing a client-server architecture where a central server manages connected clients and distributes messages instantly while maintaining stable communication.

Expected Users

- Java learners
- Networking students
- Software engineering students
- Programming instructors

---

3. Features

3.1 Server Initialization

The server starts on a specified port and waits for incoming client connections.

3.2 Client Connection

Multiple clients can connect to the server using IP Address and Port Number.

3.3 User Registration

Each connected client enters a username before joining the chat session.

3.4 Real-Time Messaging

Users can send text messages that are instantly delivered to connected users.

3.5 Broadcast Messaging

The server broadcasts incoming messages to all connected clients.

3.6 Client Disconnection

Users may leave the chat safely.

The server automatically removes disconnected clients.

3.7 Connection Status

Display:

- Connected Users
- User Joined
- User Left
- Server Status

3.8 Exception Handling

The application handles:

- Network interruptions
- Invalid connections
- Client disconnections
- Input errors
- Socket exceptions

---

4. Functional Requirements

The system shall:

FR-01 Start a chat server.

FR-02 Accept multiple client connections.

FR-03 Register connected users.

FR-04 Exchange messages in real time.

FR-05 Broadcast messages to connected clients.

FR-06 Detect client disconnections.

FR-07 Remove inactive clients.

FR-08 Display connection status.

FR-09 Handle communication errors gracefully.

FR-10 Support concurrent communication using multithreading.

---

5. Non-Functional Requirements

Performance

The server should process multiple client messages with minimal delay.

Reliability

Communication should remain stable while multiple users are connected.

Scalability

The server should support additional clients without redesigning the application.

Maintainability

Networking operations should be organized into reusable classes.

Availability

The server should remain operational until intentionally stopped.

Security

The system should validate user input and handle unexpected network behavior safely.

---

6. Assumptions and Constraints

Assumptions

- All clients connect through the same network.
- Every user has a unique username.
- The server starts before clients connect.
- Communication uses TCP sockets.

Constraints

- Java Console Application
- Java JDK 21 or later
- TCP Socket Programming
- Local Area Network (LAN) or Localhost
- No database
- No encryption
- No file sharing

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Design server, client, and message classes.
Socket Programming	Enable communication between clients and server.
Networking	Exchange data across connected systems.
Multithreading	Handle multiple clients simultaneously without blocking the server.
Collections (`ArrayList`)	Store active client connections dynamically.
Generics	Ensure type-safe management of connected clients.
Exception Handling	Handle network failures and invalid connections.
Input/Output Streams	Send and receive messages between server and clients.
Methods	Separate networking operations into reusable modules.
Loops	Continuously receive and transmit messages.
Scanner Class	Accept user messages from the console.
Modular Programming	Improve readability, scalability, and maintainability.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Build complete client-server applications.
- Understand TCP/IP communication fundamentals.
- Develop real-time communication software.
- Apply Socket Programming effectively.
- Use Multithreading to handle concurrent users.
- Design distributed applications using Object-Oriented Programming.
- Handle networking exceptions professionally.
- Manage multiple client connections using Collections.
- Strengthen software architecture and concurrent programming skills.
- Build a strong foundation for distributed systems and network applications.

Conclusion

The **Chat Application** is an above-intermediate Java project that introduces real-time client-server communication using Socket Programming and Multithreading. By implementing message broadcasting, client management, and concurrent networking, learners gain practical experience in developing distributed applications. This project serves as an excellent foundation

for advanced software such as Messaging Platforms, Collaboration Tools, Multiplayer Games, Video Conferencing Systems, and Enterprise Communication Applications.

Software Requirements Specification (SRS)

Phase 3: Project 12

Hotel Reservation System

Project Type: Java Console Application (Database-Driven)
Difficulty Level: ⭐⭐⭐⭐ Above Intermediate
Programming Language: Java
Database: MySQL
Connectivity: JDBC
Architecture: MVC (Model-View-Controller)

---

1. Project Overview

1.1 Introduction

The **Hotel Reservation System** is a Java database-driven application designed to automate hotel room reservations and customer management. The system enables users to register guests, manage room information, check room availability, book rooms, cancel reservations, and generate reservation reports.

Unlike previous projects where data was stored in memory or local files, this application stores all information in a **MySQL database** using **JDBC (Java Database Connectivity)**. The project also introduces the **MVC (Model-View-Controller)** architectural pattern, making the application modular, scalable, and easier to maintain.

1.2 Purpose

The purpose of this project is to develop a professional reservation system that manages hotel bookings efficiently while introducing learners to database programming, SQL, JDBC, and MVC architecture.

1.3 Objectives

The objectives of this project are to:

- Manage guest information.
- Maintain hotel room records.
- Check room availability.
- Book hotel rooms.
- Cancel reservations.
- Generate reservation reports.
- Store data permanently using MySQL.
- Access the database using JDBC.
- Design the application using MVC architecture.

1.4 Scope

The Hotel Reservation System provides a console-based interface for hotel management.

The system includes:

- Guest Management
- Room Management
- Reservation Management
- Booking Cancellation
- Room Availability
- Database Storage
- Report Generation

The application does not include online payments, room service, employee management, or web interfaces.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- Hotel Management Systems
- Resort Reservation Software
- Guest House Management
- Hostel Management Systems
- Online Hotel Booking Platforms
- Hospitality ERP Solutions

---

2. Problem Statement

Hotels manage hundreds of rooms and reservations daily. Manual reservation systems often lead to duplicate bookings, inaccurate room availability, and inefficient customer management.

The **Hotel Reservation System** automates hotel operations by maintaining guest records, room details, and reservation information in a centralized database. It provides accurate booking management while reducing manual effort and improving operational efficiency.

Expected Users

- Hotel Receptionists
- Hotel Managers
- Hospitality Businesses
- Java Learners
- Programming Instructors

---

3. Features

3.1 Guest Management

Register and manage guest information including:

- Guest ID
- Guest Name
- Contact Number
- Email Address

3.2 Room Management

Maintain room information such as:

- Room Number
- Room Type
- Price Per Night
- Availability Status

3.3 Room Availability

Users can search available rooms based on:

- Room Type
- Availability Status

3.4 Room Reservation

Book available rooms by selecting:

- Guest
- Room
- Check-in Date
- Check-out Date

The system updates room availability automatically.

3.5 Reservation Cancellation

Users can cancel existing reservations.

The system restores room availability after cancellation.

3.6 Reservation Reports

Display reservation information including:

- Reservation ID
- Guest Details
- Room Details
- Booking Dates
- Reservation Status

3.7 Database Storage

Store all information permanently in a MySQL database using JDBC.

---

4. Functional Requirements

The system shall:

FR-01 Register guests.

FR-02 Manage room information.

FR-03 Display available rooms.

FR-04 Book hotel rooms.

FR-05 Cancel reservations.

FR-06 Generate reservation reports.

FR-07 Store records in MySQL.

FR-08 Retrieve records using JDBC.

FR-09 Validate all user inputs.

FR-10 Handle database and runtime exceptions.

---

5. Non-Functional Requirements

Performance

The application should retrieve and update reservation records efficiently.

Reliability

Database transactions should maintain accurate reservation information.

Usability

The system should provide a simple menu-driven interface.

Maintainability

The project should follow the MVC architecture for easier maintenance and future enhancements.

Scalability

The application should support increasing numbers of guests, rooms, and reservations.

Data Persistence

All reservation data should be stored permanently in the MySQL database.

Security

The system should validate all inputs and prevent invalid database operations.

---

6. Assumptions and Constraints

Assumptions

- Every guest has a unique Guest ID.
- Every room has a unique Room Number.
- A room cannot be booked by multiple guests for the same period.
- MySQL Server is available and properly configured.
- JDBC Driver is installed.

Constraints

- Java Console Application
- Java JDK 21 or later
- MySQL Database
- JDBC Connectivity
- MVC Architecture
- Single-user application

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Model guests, rooms, and reservations as separate classes.
JDBC	Connect Java applications with the MySQL database.
MySQL	Store hotel, guest, and reservation records permanently.
SQL	Perform INSERT, UPDATE, DELETE, and SELECT operations.
MVC Architecture	Separate application into Model, View, and Controller layers.
Collections (`ArrayList`)	Temporarily manage retrieved records before displaying them.
Exception Handling	Handle database connection failures and SQL errors.
File Handling (Optional)	Store configuration files such as database properties.
Generics	Ensure type-safe collection management.
Methods	Organize reservation operations into reusable modules.
Loops	Process retrieved database records.
Conditional Statements	Validate bookings and room availability.
Scanner Class	Accept user input through the console.
Modular Programming	Improve code organization and maintainability.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop database-driven Java applications.
- Connect Java programs to MySQL using JDBC.
- Perform CRUD operations using SQL.
- Design software using the MVC architecture.

- Build scalable reservation systems.
- Handle SQL and runtime exceptions effectively.
- Work with relational databases in Java.
- Apply Object-Oriented Programming to enterprise applications.
- Understand transaction-based application development.
- Build a strong foundation for enterprise Java backend development.

Conclusion

The **Hotel Reservation System** is an above-intermediate Java project that introduces professional software development practices by integrating Java with MySQL through JDBC while following the MVC architectural pattern. By implementing guest management, room reservations, availability tracking, and database persistence, learners gain practical experience in building scalable, maintainable, and database-driven applications. This project serves as an excellent foundation for developing enterprise hotel management software, booking platforms, hospital management systems, airline reservation systems, and other business-critical applications.

Software Requirements Specification (SRS)

Phase 3: Project 13

Food Ordering System

Project Type: Java Console Application (Database-Driven)
Difficulty Level: ⋆ ⋆ ⋆ ⋆ Above Intermediate
Programming Language: Java
Database: MySQL
Connectivity: JDBC
Architecture: MVC (Model-View-Controller)

---

1. Project Overview

1.1 Introduction

The **Food Ordering System** is a Java database-driven application designed to automate the process of ordering food from a restaurant. The system allows customers to browse the menu, add food items to a cart, place orders, cancel orders, and view order history. Restaurant administrators can manage menu items and monitor customer orders.

The application stores all data in a **MySQL database** using **JDBC (Java Database Connectivity)** and follows the **MVC (Model-View-Controller)** architecture to ensure clean separation of business logic, user interface, and data management.

1.2 Purpose

The purpose of this project is to develop a digital food ordering platform that simplifies restaurant order management while introducing learners to database programming, SQL, MVC architecture, and enterprise Java application development.

1.3 Objectives

The objectives of this project are to:

- Manage restaurant menu items.
- Register customers.
- Allow customers to place food orders.
- Maintain shopping cart functionality.
- Calculate order totals automatically.
- Generate order summaries.
- Store all records in MySQL.
- Practice JDBC and MVC architecture.

1.4 Scope

The Food Ordering System provides a console-based platform for restaurant order management.

The system includes:

- Customer Management
- Menu Management
- Cart Management
- Order Placement
- Order Cancellation

- Bill Generation
- Database Storage
- Order History

The application does not include online payment gateways, food delivery tracking, mobile applications, or customer authentication.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- Restaurant Management Systems
- Online Food Delivery Platforms
- Cafeteria Management Systems
- Hotel Ordering Systems
- Cloud Kitchen Software
- POS (Point of Sale) Systems

---

2. Problem Statement

Restaurants often struggle with manual order management, leading to incorrect orders, delayed service, and inefficient billing. Paper-based systems become difficult to manage as the number of customers increases.

The **Food Ordering System** automates menu management, customer ordering, bill calculation, and order tracking, improving operational efficiency and reducing human errors.

Expected Users

- Restaurant Owners
- Restaurant Staff
- Customers
- Java Learners
- Programming Instructors

---

3. Features

3.1 Customer Management

Register customer information including:

- Customer ID
- Customer Name
- Phone Number
- Address (Optional)

3.2 Menu Management

Maintain restaurant menu items including:

- Food ID
- Food Name
- Category
- Price
- Availability Status

3.3 Browse Menu

Customers can view available food items with prices and categories.

3.4 Shopping Cart

Customers can:

- Add food items
- Remove food items
- Update quantities
- View cart contents

The system automatically calculates the running total.

3.5 Place Order

Customers can place orders directly from the cart.

The system:

- Stores the order
- Calculates the total bill
- Updates order status

3.6 Cancel Order

Customers can cancel orders before they are processed.

3.7 Order History

Display previous orders including:

- Order ID
- Customer Name
- Ordered Items
- Quantity
- Total Amount
- Order Status

3.8 Database Storage

Store customer details, menu items, cart information, and order records permanently using MySQL.

---

4. Functional Requirements

The system shall:

FR-01 Register customers.

FR-02 Manage food menu items.

FR-03 Display available menu items.

FR-04 Add items to the shopping cart.

FR-05 Remove items from the cart.

FR-06 Place food orders.

FR-07 Cancel food orders.

FR-08 Calculate total bill automatically.

FR-09 Display order history.

FR-10 Store records in MySQL using JDBC.

FR-11 Validate all user inputs.

FR-12 Handle runtime and database exceptions.

---

5. Non-Functional Requirements

Performance

The application should process orders and retrieve menu information quickly.

Reliability

Order information and billing calculations should remain accurate and consistent.

Usability

The system should provide a simple, menu-driven console interface.

Maintainability

The application should follow the MVC architecture for easier maintenance and future expansion.

Scalability

The system should support adding new menu items and handling increasing customer orders.

Data Persistence

Customer information, menu details, and order history should be permanently stored in MySQL.

Security

The application should validate all user input and prevent invalid database operations.

---

6. Assumptions and Constraints

Assumptions

- Every customer has a unique Customer ID.
- Every food item has a unique Food ID.
- Food prices are positive values.
- Orders are placed only for available menu items.
- MySQL Server and JDBC Driver are properly configured.

Constraints

- Java Console Application
- Java JDK 21 or later
- MySQL Database
- JDBC Connectivity
- MVC Architecture
- Single-user application

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Model customers, menu items, carts, and orders as separate classes.
JDBC	Connect Java applications to the MySQL database.
MySQL	Store customer, menu, and order information permanently.
SQL	Perform CRUD operations on database tables.
MVC Architecture	Separate application into Model, View, and Controller components.
Collections (`ArrayList`)	Manage cart items and retrieved database records.
Exception Handling	Handle invalid input and SQL-related errors safely.
Generics	Ensure type-safe collection management.
Comparable	Sort menu items by ID or Name.
Comparator	Sort food items by price or category.
Lambda Expressions	Simplify sorting, filtering, and searching operations.
Methods	Organize ordering operations into reusable modules.
Loops	Process menu items and customer orders.
Conditional Statements	Validate orders and food availability.
Scanner Class	Accept user input through the console.
Modular Programming	Improve readability, maintainability, and scalability.

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop complete database-driven Java applications.
- Build CRUD-based restaurant management software.
- Connect Java applications with MySQL using JDBC.
- Apply SQL queries to manage application data.
- Implement the MVC architectural pattern.
- Design shopping cart and order management systems.
- Handle database and runtime exceptions professionally.
- Develop modular and scalable Java software.
- Understand business workflows in restaurant management systems.
- Build a strong foundation for enterprise e-commerce and ordering applications.

Conclusion

The **Food Ordering System** is an above-intermediate Java project that combines Object-Oriented Programming, JDBC, MySQL, SQL, and the MVC architectural pattern to build a practical restaurant ordering application. By implementing menu management, shopping cart functionality, order processing, billing, and persistent database storage, learners gain hands-on experience in developing enterprise-style software. This project serves as an excellent foundation for building online food delivery platforms, restaurant management systems, e-commerce applications, and other database-driven business solutions.

Software Requirements Specification (SRS)

Phase 3: Project 14

Online Examination System

Project Type: Java Console Application (Database-Driven)
Difficulty Level: ⭑ ⭑ ⭑ ⭑ Above Intermediate
Programming Language: Java
Database: MySQL
Connectivity: JDBC
Architecture: MVC (Model-View-Controller)

---

1. Project Overview

1.1 Introduction

The **Online Examination System** is a Java database-driven application developed to automate the process of conducting examinations, evaluating answers, and managing student results. The system enables administrators to manage subjects and questions, while students can log in, take examinations, submit answers, and view their results.

The application stores all examination data in a **MySQL database** using **JDBC (Java Database Connectivity)** and follows the **MVC (Model-View-Controller)** architecture to separate business logic, presentation, and data access layers.

This project closely resembles real-world online examination platforms used by schools, colleges, universities, and certification organizations.

1.2 Purpose

The purpose of this project is to replace traditional paper-based examinations with a computerized examination system that automatically evaluates answers, calculates scores, and stores examination records securely.

1.3 Objectives

The objectives of this project are to:

- Manage student records.
- Manage examination subjects.
- Manage multiple-choice questions.
- Conduct online examinations.
- Evaluate answers automatically.
- Generate scorecards and reports.
- Store examination data in MySQL.
- Practice JDBC and MVC architecture.

1.4 Scope

The Online Examination System provides a console-based environment for conducting objective examinations.

The system includes:

- Student Management
- Subject Management
- Question Management
- Examination Management
- Automatic Evaluation
- Result Generation
- Database Storage
- Examination Reports

The application does not include webcam monitoring, AI-based proctoring, descriptive answer evaluation, or online video streaming.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- Online Examination Platforms
- Learning Management Systems (LMS)
- University Examination Portals
- Recruitment Assessment Systems
- Certification Platforms
- Employee Skill Assessment Systems

---

2. Problem Statement

Traditional examination systems require manual paper preparation, answer evaluation, and result generation, making the process time-consuming and prone to human error.

The **Online Examination System** automates examination management by storing questions digitally, conducting examinations electronically, evaluating answers instantly, and generating accurate results.

Expected Users

- Educational Institutions
- Schools
- Colleges
- Universities
- Training Institutes
- Java Learners
- Programming Instructors

---

3. Features

3.1 Student Management

Register and manage student information including:

- Student ID
- Student Name
- Email

- Course

3.2 Subject Management

Manage examination subjects such as:

- Subject ID
- Subject Name
- Description

3.3 Question Management

Administrators can add and manage:

- Question ID
- Question Statement
- Four Options
- Correct Answer
- Subject

3.4 Start Examination

Students can:

- Select a subject
- Start the examination
- Answer multiple-choice questions
- Submit the examination

3.5 Automatic Evaluation

The system automatically:

- Compares submitted answers
- Calculates marks
- Calculates percentage

- Determines Pass/Fail status
- Generates final score

3.6 Result Management

Display:

- Student Name
- Subject
- Total Questions
- Correct Answers
- Incorrect Answers
- Marks Obtained
- Percentage
- Pass/Fail Status

3.7 Database Storage

Store student records, questions, examinations, and results permanently using MySQL.

---

4. Functional Requirements

The system shall:

FR-01 Register students.

FR-02 Manage examination subjects.

FR-03 Add examination questions.

FR-04 Conduct online examinations.

FR-05 Evaluate answers automatically.

FR-06 Generate examination results.

FR-07 Display student scorecards.

FR-08 Store examination records using MySQL.

FR-09 Retrieve records using JDBC.

FR-10 Validate all user inputs.

FR-11 Handle database and runtime exceptions.

---

5. Non-Functional Requirements

Performance

The application should evaluate examinations and generate results immediately after submission.

Reliability

The system should produce accurate evaluation results consistently.

Usability

The console interface should be simple, structured, and easy to navigate.

Maintainability

The project should follow the MVC architecture for easier maintenance and feature expansion.

Scalability

The system should support increasing numbers of students, subjects, and questions.

Data Persistence

Student records, examination data, questions, and results should remain permanently stored in MySQL.

Security

The application should validate all user inputs and prevent invalid database operations.

---

6. Assumptions and Constraints

Assumptions

- Every student has a unique Student ID.
- Every subject has a unique Subject ID.
- Every question has a unique Question ID.
- Each question has one correct answer.
- MySQL Server and JDBC Driver are configured correctly.

Constraints

- Java Console Application
- Java JDK 21 or later
- MySQL Database
- JDBC Connectivity
- MVC Architecture
- Single-user application

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Represent students, subjects, questions, examinations, and results as separate classes.
JDBC	Connect the application to the MySQL database.

Java Concept	Application in the Project
MySQL	Store student, examination, and result information permanently.
SQL	Perform CRUD operations on examination-related tables.
MVC Architecture	Separate business logic, user interface, and database access.
Collections (`ArrayList`)	Store retrieved questions and examination results temporarily.
Exception Handling	Handle invalid input, database failures, and runtime exceptions.
Generics	Ensure type-safe collection management.
Comparable	Sort students or subjects naturally by ID or name.
Comparator	Sort examination results based on marks or percentage.
Lambda Expressions	Simplify sorting, filtering, and report generation.
Methods	Organize examination operations into reusable modules.
Loops	Display questions and process student answers.
Conditional Statements	Evaluate answers and determine Pass/Fail status.
Scanner Class	Accept examination responses through the console.
Modular Programming	Improve maintainability and scalability of the application.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop a complete database-driven examination system.
- Build applications using the MVC architecture.
- Integrate Java with MySQL using JDBC.
- Perform CRUD operations using SQL.
- Design automatic evaluation algorithms.
- Implement result generation and score reporting.
- Handle exceptions and database transactions effectively.
- Develop modular and maintainable enterprise applications.
- Understand examination management workflows.
- Build a strong foundation for educational software development.

Conclusion

The **Online Examination System** is an above-intermediate Java project that integrates Object-Oriented Programming, JDBC, MySQL, SQL, and the MVC architectural pattern to build a professional examination platform. By implementing student management, question management, online examinations, automatic evaluation, result generation, and persistent database storage, learners gain practical experience in developing enterprise-grade educational

software. This project serves as a strong foundation for building Learning Management Systems (LMS), Computer-Based Testing (CBT) platforms, university examination portals, and large-scale assessment systems.

Software Requirements Specification (SRS)

Phase 3: Project 15

College ERP System (Mini Version)

Project Type: Java Console Application (Enterprise Management System)
Difficulty Level: ⭑ ⭑ ⭑ ⭑ ⭑ Advanced
Programming Language: Java
Database: MySQL
Connectivity: JDBC
Architecture: MVC (Model-View-Controller)

---

1. Project Overview

1.1 Introduction

The **College ERP (Enterprise Resource Planning) System – Mini Version** is a Java-based enterprise management application designed to automate the core administrative and academic activities of a college. The system integrates multiple modules into a single application, allowing efficient management of students, faculty, courses, departments, attendance, examinations, and academic records.

Unlike previous projects that focus on a single business process, this project combines several interconnected modules into one integrated software system. It demonstrates how enterprise applications are designed using **Object-Oriented Programming**, **JDBC**, **MySQL**, and the **MVC (Model-View-Controller)** architecture.

This project closely resembles the structure of real ERP systems used in universities and educational institutions.

1.2 Purpose

The purpose of this project is to develop a centralized academic management system that stores and manages college-related information efficiently while introducing learners to enterprise-level software architecture and database-driven application development.

1.3 Objectives

The objectives of this project are to:

- Manage student records.
- Manage faculty records.
- Manage departments and courses.
- Maintain attendance records.
- Store examination results.
- Generate academic reports.
- Integrate multiple modules into one application.
- Store data permanently using MySQL.
- Apply MVC architecture and JDBC.

1.4 Scope

The College ERP System provides a console-based management platform.

The system includes:

- Student Management
- Faculty Management
- Department Management
- Course Management
- Attendance Management
- Examination Management
- Result Management
- Database Storage
- Report Generation

The application does not include online learning, fee payment gateways, biometric attendance, hostel management, or library management.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- College ERP Systems
- University Management Systems
- School Management Software
- Student Information Systems (SIS)
- Campus Administration Platforms
- Educational ERP Solutions

---

2. Problem Statement

Educational institutions manage thousands of students, faculty members, academic records, attendance, and examination data. Managing these activities manually often results in data duplication, inaccurate records, delayed report generation, and inefficient administration.

The **College ERP System** integrates academic and administrative operations into a single software application, enabling centralized data management, faster information retrieval, and improved operational efficiency.

Expected Users

- College Administrators
- Faculty Members
- Department Coordinators
- Academic Staff
- Java Learners
- Programming Instructors

---

3. Features

3.1 Student Management

Manage student information including:

- Student ID
- Name
- Department
- Course
- Semester
- Contact Information

3.2 Faculty Management

Maintain faculty records such as:

- Faculty ID
- Faculty Name
- Department
- Designation
- Contact Information

3.3 Department Management

Manage departments including:

- Department ID
- Department Name
- Head of Department

3.4 Course Management

Manage academic courses including:

- Course ID
- Course Name
- Credits
- Assigned Faculty

3.5 Attendance Management

Record and maintain student attendance.

Display:

- Total Classes
- Present
- Absent

- Attendance Percentage

3.6 Examination & Result Management

Maintain examination records including:

- Subject
- Internal Marks
- External Marks
- Total Marks
- Grade
- Pass/Fail Status

3.7 Search and Reports

Users can search:

- Students
- Faculty
- Departments
- Courses

Generate reports such as:

- Student Report
- Faculty Report
- Attendance Report
- Examination Report

3.8 Database Storage

Store all academic and administrative information permanently in a MySQL database using JDBC.

---

4. Functional Requirements

The system shall:

FR-01 Register and manage students.

FR-02 Register and manage faculty.

FR-03 Manage departments.

FR-04 Manage academic courses.

FR-05 Record attendance.

FR-06 Manage examination records.

FR-07 Generate academic reports.

FR-08 Store data in MySQL.

FR-09 Retrieve records using JDBC.

FR-10 Validate all user inputs.

FR-11 Handle database and runtime exceptions.

---

5. Non-Functional Requirements

Performance

The application should retrieve and process academic records efficiently.

Reliability

Academic information should remain accurate and consistent across all modules.

Usability

The system should provide a structured, menu-driven console interface.

Maintainability

The application should follow the MVC architecture and modular programming practices for easier maintenance.

Scalability

The design should support future modules such as fee management, hostel management, and library management.

Data Persistence

All academic records should remain permanently stored in the MySQL database.

Security

The system should validate all user input and prevent invalid database transactions.

---

6. Assumptions and Constraints

Assumptions

- Every student has a unique Student ID.
- Every faculty member has a unique Faculty ID.
- Every department has a unique Department ID.
- Every course has a unique Course ID.
- Attendance and examination records are associated with valid students.
- MySQL Server and JDBC Driver are correctly configured.

Constraints

- Java Console Application
- Java JDK 21 or later
- MySQL Database
- JDBC Connectivity
- MVC Architecture
- Single-user application

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Model students, faculty, departments, courses, attendance, and examinations as separate classes.
JDBC	Connect the application to the MySQL database.
MySQL	Store institutional records permanently.
SQL	Perform CRUD operations across multiple related database tables.
MVC Architecture	Separate application into Model, View, and Controller layers.
Collections (`ArrayList`)	Temporarily manage retrieved records before displaying them.
Exception Handling	Handle invalid input, SQL errors, and runtime exceptions.
Generics	Ensure type-safe collection management.
Comparable	Sort students or faculty by ID or name.
Comparator	Sort records by department, attendance percentage, or marks.
Lambda Expressions	Simplify filtering, sorting, and report generation.
Methods	Organize business logic into reusable modules.
Loops	Process records retrieved from the database.
Conditional Statements	Validate academic rules and business logic.
Scanner Class	Accept administrator input through the console.
Modular Programming	Improve maintainability, scalability, and readability.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop a complete enterprise-style Java application.
- Design software consisting of multiple interconnected modules.
- Apply the MVC architecture in large applications.
- Integrate Java with MySQL using JDBC.
- Design normalized database structures.
- Perform CRUD operations across multiple related entities.

- Build modular and maintainable software systems.
- Handle exceptions and database transactions effectively.
- Understand enterprise software architecture.
- Build a strong foundation for developing ERP systems and large-scale management software.

Conclusion

The **College ERP System (Mini Version)** is the capstone project of the Above Intermediate phase. It combines multiple academic management modules into a single enterprise application using Object-Oriented Programming, JDBC, MySQL, SQL, and the MVC architectural pattern. By implementing student management, faculty administration, attendance tracking, course management, examination processing, and report generation, learners gain practical experience in building integrated enterprise software. This project provides a strong bridge to advanced Java technologies such as Spring Boot, Hibernate, REST APIs, and full-stack enterprise application development.

Software Requirements Specification (SRS)

Phase 4: Project 16

E-Commerce Backend

Project Type: Java Backend Application
Difficulty Level: ⭑ ⭑ ⭑ ⭑ ⭑ Professional
Programming Language: Java
Build Tool: Maven / Gradle
Database: MySQL
Connectivity: JDBC (or JPA in future enhancements)
Architecture: Clean Architecture (MVC-compatible)

---

1. Project Overview

1.1 Introduction

The **E-Commerce Backend** is a professional Java backend application designed to manage the core functionalities of an online shopping platform. The system handles product management, customer management, shopping carts, orders, inventory, and order processing through a modular and scalable architecture.

Unlike previous console-based management systems, this project emphasizes **software architecture**, **maintainability**, **code quality**, and **professional development practices**. The application follows **SOLID principles**, **Design Patterns**, **Clean Architecture**, and utilizes **Maven/Gradle** for dependency management, **Logging** for monitoring, and **Unit Testing** to ensure software reliability.

1.2 Purpose

The purpose of this project is to simulate the backend of a real-world e-commerce platform while teaching learners how enterprise software is structured and maintained.

1.3 Objectives

The objectives of this project are to:

- Manage customers.
- Manage products and inventory.
- Handle shopping carts.
- Process customer orders.
- Generate invoices.
- Maintain order history.
- Apply professional software engineering practices.
- Build scalable and maintainable backend systems.

1.4 Scope

The E-Commerce Backend manages all core business operations of an online shopping platform.

The system includes:

- Customer Management
- Product Management
- Category Management
- Inventory Management

- Shopping Cart
- Order Processing
- Invoice Generation
- Database Storage
- Logging
- Testing

Online payment gateways, shipping integration, notifications, and REST APIs are outside the scope of this project.

1.5 Real-World Applications

The concepts implemented in this project are used in:

- Amazon
- Flipkart
- eBay
- Shopify
- Walmart
- Enterprise E-Commerce Platforms
- Retail ERP Systems

---

2. Problem Statement

Managing products, customers, inventory, and orders manually becomes increasingly difficult as business grows. Maintaining accurate stock information, processing customer orders, and generating invoices require a reliable backend system.

The **E-Commerce Backend** automates these operations by providing a structured backend capable of managing products, customers, shopping carts, and orders while ensuring maintainability and scalability.

Expected Users

- Online Retailers
- Store Administrators
- Warehouse Managers
- Java Backend Developers
- Software Engineering Students

---

3. Features

3.1 Customer Management

Manage customer information including:

- Customer ID
- Name
- Email
- Contact Number
- Address

3.2 Product Management

Manage products including:

- Product ID
- Product Name
- Category
- Price
- Stock Quantity

3.3 Category Management

Create and organize products into categories.

3.4 Shopping Cart

Allow customers to:

- Add products
- Remove products
- Update quantities
- View cart summary

3.5 Order Processing

The system:

- Validates stock availability.
- Creates customer orders.
- Updates inventory automatically.
- Generates order confirmation.

3.6 Invoice Generation

Generate invoices including:

- Order ID
- Customer Details
- Purchased Products
- Quantity
- Total Amount

3.7 Inventory Management

Automatically update stock levels after every successful order.

3.8 Logging

Record:

- User operations
- System events
- Errors
- Order processing logs

3.9 Unit Testing

Verify:

- Business logic
- Order calculations
- Inventory updates
- Cart operations

---

4. Functional Requirements

The system shall:

FR-01 Register customers.

FR-02 Manage product catalog.

FR-03 Manage product categories.

FR-04 Manage shopping carts.

FR-05 Process customer orders.

FR-06 Generate invoices.

FR-07 Update inventory automatically.

FR-08 Store application data in MySQL.

FR-09 Record application logs.

FR-10 Support automated unit testing.

FR-11 Validate all business rules.

FR-12 Handle runtime exceptions.

---

5. Non-Functional Requirements

Performance

The system should process customer orders efficiently.

Reliability

Inventory and order information should remain accurate during transactions.

Maintainability

The application should follow Clean Architecture and SOLID principles.

Scalability

The system should support future REST APIs, payment gateways, and microservices.

Security

Validate all inputs and protect business logic from invalid operations.

Logging

Record important application events for monitoring and debugging.

Testability

The business logic should be independently testable using unit tests.

---

6. Assumptions and Constraints

Assumptions

- Every customer has a unique Customer ID.
- Every product has a unique Product ID.

- Stock quantity cannot become negative.
- Orders can only be placed when inventory is available.
- MySQL and Maven/Gradle are properly configured.

Constraints

- Java Backend Application
- Java JDK 21 or later
- MySQL Database
- Maven or Gradle Build System
- Clean Architecture
- SOLID Principles
- Single-node deployment

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Model customers, products, orders, and inventory.
SOLID Principles	Build loosely coupled and maintainable software.
Design Patterns	Apply Factory, Singleton, Strategy, DAO, and Builder patterns where appropriate.
Clean Architecture	Separate business logic, data access, and presentation layers.
Maven / Gradle	Manage dependencies and project builds.
JDBC / MySQL	Store and retrieve business data.
Collections & Generics	Manage products, carts, and orders efficiently.
Exception Handling	Handle business and runtime errors gracefully.
Logging	Record application activities and errors.
Unit Testing	Verify business logic using automated tests.
Modular Programming	Organize the project into reusable packages and classes.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Build enterprise-style backend applications.

- Apply SOLID principles effectively.
- Implement common Design Patterns.
- Organize projects using Clean Architecture.
- Manage dependencies using Maven or Gradle.
- Develop maintainable and scalable Java software.
- Implement structured logging for debugging and monitoring.
- Write unit tests for business logic.
- Design backend systems for real-world e-commerce applications.
- Build a strong foundation for Spring Boot and enterprise Java development.

Conclusion

The **E-Commerce Backend** is the first professional-level Java project in this roadmap. It introduces enterprise software engineering concepts such as SOLID principles, Design Patterns, Clean Architecture, Maven/Gradle, Logging, and Unit Testing while implementing a realistic online shopping backend. This project prepares learners for developing production-quality backend systems using modern Java frameworks such as Spring Boot, Hibernate, REST APIs, and Microservices.

Software Requirements Specification (SRS)

Project 17

URL Shortener (Bit.ly Clone)

Project Type: Java Backend Application
Difficulty Level: ⭑ ⭑ ⭑ ⭑ ⭑ Professional
Programming Language: Java
Build Tool: Maven / Gradle
Database: MySQL
Connectivity: JDBC (Future Enhancement: JPA/Hibernate)
Architecture: Clean Architecture

---

1. Project Overview

1.1 Introduction

The **URL Shortener** is a Java backend application that converts long URLs into short, unique, and easily shareable links. When users access a shortened URL, the system retrieves the original URL from the database and redirects the user to the intended destination.

The project demonstrates how high-performance backend services are designed by implementing URL generation, database persistence, click tracking, analytics, logging, testing, and clean software architecture.

This project is inspired by services such as **Bit.ly**, **TinyURL**, and **Short.io**.

1.2 Purpose

The purpose of this project is to build a scalable URL shortening service that efficiently generates unique short links, stores them securely, tracks usage statistics, and demonstrates professional backend development practices.

1.3 Objectives

The objectives of this project are to:

- Generate unique short URLs.
- Store original and shortened URLs.
- Redirect users to original URLs.
- Track click statistics.
- Manage URL records.
- Apply Clean Architecture and SOLID principles.
- Practice professional backend development.

1.4 Scope

The URL Shortener provides backend services for URL management.

The system includes:

- URL Shortening
- URL Redirection
- URL Search
- Click Tracking
- URL Analytics

- Database Storage
- Logging
- Unit Testing

The application does not include user authentication, custom domains, QR code generation, or distributed caching.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- Bit.ly
- TinyURL
- Short.io
- Marketing Platforms
- Social Media Platforms
- Analytics Systems
- Digital Advertising Services

---

2. Problem Statement

Long URLs are difficult to remember, share, and manage, especially on social media, emails, and marketing campaigns. Businesses also require analytics to measure how frequently links are accessed.

The **URL Shortener** solves this problem by generating compact URLs that redirect users to the original destination while collecting useful statistics such as click counts.

Expected Users

- Digital Marketers
- Businesses
- Social Media Users
- Java Backend Developers
- Software Engineering Students

---

3. Features

3.1 URL Shortening

Users can submit a long URL.

The system automatically:

- Validates the URL
- Generates a unique short code
- Creates a shortened URL
- Stores both URLs

3.2 URL Redirection

When a user opens a shortened URL:

- The system searches the database.
- Retrieves the original URL.
- Redirects the user automatically.

3.3 URL Management

Users can:

- View shortened URLs
- Search URLs
- Delete URLs
- Update URL information (optional)

3.4 Click Tracking

Every visit updates:

- Click Count
- Last Access Time

3.5 URL Analytics

Generate reports showing:

- Original URL
- Short URL
- Total Clicks
- Creation Date
- Last Access Date

3.6 Logging

Record:

- URL creation
- URL access
- Errors
- Database operations

3.7 Unit Testing

Test:

- URL generation
- Redirection logic
- Database operations
- Analytics calculations

---

4. Functional Requirements

The system shall:

FR-01 Accept long URLs.

FR-02 Generate unique short URLs.

FR-03 Store URL mappings.

FR-04 Redirect users to original URLs.

FR-05 Count URL visits.

FR-06 Display URL analytics.

FR-07 Delete URL records.

FR-08 Store information in MySQL.

FR-09 Generate application logs.

FR-10 Support automated testing.

FR-11 Validate all URLs.

FR-12 Handle runtime exceptions.

---

5. Non-Functional Requirements

Performance

URL redirection should occur with minimal delay.

Reliability

The application should consistently redirect users to the correct destination.

Scalability

The architecture should support millions of URL records in future enhancements.

Maintainability

The project should follow SOLID principles and Clean Architecture.

Security

Only valid URLs should be accepted, and invalid or malformed URLs should be rejected.

Logging

Important operations and failures should be logged for monitoring and debugging.

Testability

Business logic should be independently testable using unit tests.

---

6. Assumptions and Constraints

Assumptions

- Every short URL is unique.
- Original URLs are valid.
- Database connectivity is always available.
- MySQL and Maven/Gradle are configured correctly.

Constraints

- Java Backend Application
- Java JDK 21 or later
- MySQL Database
- Maven or Gradle
- Clean Architecture
- SOLID Principles

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Model URL records and analytics.
SOLID Principles	Design loosely coupled and maintainable components.
Design Patterns	Implement Factory, Singleton, DAO, Builder, and Strategy patterns.
Clean Architecture	Separate business rules, database access, and presentation logic.
Maven / Gradle	Manage project dependencies and builds.
JDBC / MySQL	Store URL mappings and analytics.
Collections & Generics	Manage URL objects efficiently.
Exception Handling	Handle invalid URLs and database failures.
Logging	Record URL creation, redirection, and errors.
Unit Testing	Verify URL generation and redirection logic.
Modular Programming	Organize application into reusable layers and packages.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Build scalable backend services.
- Design efficient unique ID generation algorithms.
- Apply SOLID principles and Design Patterns.
- Develop applications using Clean Architecture.
- Manage projects with Maven or Gradle.
- Store and retrieve application data using MySQL.
- Implement structured logging.
- Write automated unit tests.
- Design analytics-based backend applications.
- Understand the architecture of modern URL shortening platforms.

Conclusion

The **URL Shortener (Bit.ly Clone)** is a professional-level backend project that demonstrates how modern web services generate, store, and manage shortened URLs efficiently. By implementing unique URL generation, redirection, click tracking, analytics, logging, testing, and Clean Architecture, learners gain practical experience in building scalable backend systems. This project serves as an excellent foundation for distributed systems, REST APIs, cloud-native applications, and high-performance backend services.

Software Requirements Specification (SRS)

Project Name

Task Management System (Trello Backend)

Project Type: Java Backend Application
Difficulty Level: ⭐⭐⭐⭐⭐ Professional
Programming Language: Java
Build Tool: Maven / Gradle
Database: MySQL
Connectivity: JDBC (Future Enhancement: JPA/Hibernate)
Architecture: Clean Architecture

---

1. Project Overview

1.1 Introduction

The **Task Management System (Trello Backend)** is a Java backend application designed to manage projects and tasks efficiently. The system enables users to create projects, organize tasks into different workflow stages, assign tasks to team members, set priorities and deadlines, and monitor task progress.

Inspired by applications such as **Trello**, **Jira**, and **Asana**, this project focuses entirely on backend business logic and demonstrates professional software engineering practices using **Clean Architecture**, **SOLID Principles**, **Design Patterns**, **Maven/Gradle**, **Logging**, and **Unit Testing**.

1.2 Purpose

The purpose of this project is to develop a scalable backend system that manages projects and task workflows while introducing learners to enterprise-level software architecture and backend design.

1.3 Objectives

The objectives of this project are to:

- Manage projects.
- Create and organize tasks.
- Assign tasks to users.
- Track task progress.
- Set priorities and deadlines.
- Store project information permanently.
- Apply professional software engineering principles.

1.4 Scope

The Task Management System provides backend services for project and task management.

The system includes:

- User Management
- Project Management
- Task Management
- Task Assignment
- Task Status Tracking
- Priority Management
- Deadline Management
- Activity Logging
- Database Storage

The application does not include notifications, real-time collaboration, file sharing, or REST APIs.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- Trello
- Jira
- Asana
- Monday.com
- ClickUp
- Notion
- Enterprise Project Management Systems

---

2. Problem Statement

Managing projects manually becomes increasingly difficult as team size and project complexity grow. Tracking task progress, assignments, priorities, and deadlines without a centralized system often results in missed deadlines and poor collaboration.

The **Task Management System** automates project organization by maintaining structured project and task information while providing an efficient workflow for managing work items.

Expected Users

- Software Development Teams
- Project Managers
- Students
- Freelancers
- Java Backend Developers
- Software Engineering Learners

---

3. Features

3.1 User Management

Manage user information including:

- User ID
- Name
- Email
- Role

3.2 Project Management

Users can:

- Create projects
- Update project details
- Delete projects
- View project information

3.3 Task Management

Create tasks with:

- Task ID
- Task Name
- Description
- Project
- Priority
- Due Date

3.4 Task Assignment

Assign tasks to team members.

Each task can have one assigned user.

3.5 Task Status Management

Move tasks between workflow stages:

- To Do
- In Progress
- Review
- Completed

3.6 Priority Management

Assign priorities such as:

- Low
- Medium
- High
- Critical

3.7 Search and Filtering

Users can search tasks using:

- Task ID
- Project
- Assigned User
- Status
- Priority

3.8 Reports

Generate reports showing:

- Total Projects
- Total Tasks
- Completed Tasks
- Pending Tasks
- Overdue Tasks

3.9 Logging

Record:

- Project creation
- Task creation
- Status updates
- Assignment changes
- System errors

3.10 Unit Testing

Test:

- Task creation
- Status updates
- Assignment logic
- Report generation

- Business rules

---

4. Functional Requirements

The system shall:

FR-01 Create and manage users.

FR-02 Create and manage projects.

FR-03 Create, update, and delete tasks.

FR-04 Assign tasks to users.

FR-05 Update task status.

FR-06 Set priorities and deadlines.

FR-07 Generate project reports.

FR-08 Store records in MySQL.

FR-09 Record application logs.

FR-10 Support automated unit testing.

FR-11 Validate business rules.

FR-12 Handle runtime exceptions.

---

5. Non-Functional Requirements

Performance

The application should efficiently manage multiple projects and tasks.

Reliability

Task information should remain accurate throughout the project lifecycle.

Maintainability

The project should follow Clean Architecture and SOLID principles.

Scalability

The system should support additional modules such as notifications, comments, attachments, and REST APIs.

Security

The application should validate all user inputs before processing.

Logging

All important system activities and errors should be recorded.

Testability

Business logic should be independently testable using unit tests.

---

6. Assumptions and Constraints

Assumptions

- Every user has a unique User ID.
- Every project has a unique Project ID.
- Every task has a unique Task ID.

- Each task belongs to one project.
- A task can have only one assigned user.
- MySQL and Maven/Gradle are configured correctly.

Constraints

- Java Backend Application
- Java JDK 21 or later
- MySQL Database
- Maven or Gradle
- Clean Architecture
- SOLID Principles

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Model users, projects, tasks, and reports as separate classes.
SOLID Principles	Design modular and maintainable backend components.
Design Patterns	Apply Factory, Builder, Singleton, DAO, Strategy, and Observer patterns where appropriate.
Clean Architecture	Separate business logic, persistence, and application layers.
Maven / Gradle	Manage project dependencies and builds.
JDBC / MySQL	Store project and task information permanently.
Collections & Generics	Manage projects and tasks efficiently.
Exception Handling	Handle invalid operations and database failures.
Logging	Record task updates, assignments, and application events.
Unit Testing	Verify business logic and workflow rules.
Modular Programming	Organize the project into scalable packages and modules.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop enterprise-level project management systems.
- Apply Clean Architecture and SOLID principles effectively.

- Implement Design Patterns in real-world scenarios.
- Manage complex business workflows.
- Build scalable backend applications.
- Write maintainable and testable Java code.
- Implement structured logging.
- Develop comprehensive unit tests.
- Understand workflow management systems.
- Build a strong foundation for enterprise backend development using Spring Boot and Microservices.

Conclusion

The **Task Management System (Trello Backend)** is a professional-level Java backend project that simulates a real-world project management platform. By implementing project management, task assignment, workflow tracking, priority management, reporting, logging, and persistent database storage, learners gain practical experience in designing scalable enterprise applications. This project prepares learners for developing modern project management software, collaborative work platforms, Agile development tools, and enterprise workflow management systems.

Software Requirements Specification (SRS)

Project Name

Ride Booking Backend (Uber Mini)

Project Type: Java Backend Application
Difficulty Level: ⭑ ⭑ ⭑ ⭑ ⭑ Professional
Programming Language: Java
Build Tool: Maven / Gradle
Database: MySQL
Connectivity: JDBC (Future Enhancement: JPA/Hibernate)
Architecture: Clean Architecture

---

1. Project Overview

1.1 Introduction

The **Ride Booking Backend (Uber Mini)** is a Java backend application that simulates the core functionality of a ride-hailing platform. The system enables passengers to request rides, drivers to accept or reject requests, and administrators to manage users and ride records. It also calculates fares, tracks ride status, and maintains ride history.

Inspired by platforms such as **Uber**, **Ola**, and **Lyft**, this project focuses on backend business logic rather than user interface development. It emphasizes professional software engineering practices using **SOLID Principles**, **Design Patterns**, **Clean Architecture**, **Maven/Gradle**, **Logging**, and **Unit Testing**.

1.2 Purpose

The purpose of this project is to build a scalable backend system capable of managing ride requests, driver allocation, fare calculation, and ride tracking while exposing learners to real-world backend engineering challenges.

1.3 Objectives

The objectives of this project are to:

- Manage passengers and drivers.
- Request and assign rides.
- Calculate ride fares.
- Track ride status.
- Maintain ride history.
- Store ride information permanently.
- Apply enterprise software engineering principles.

1.4 Scope

The Ride Booking Backend manages ride booking operations through a backend application.

The system includes:

- Passenger Management

- Driver Management
- Vehicle Management
- Ride Booking
- Ride Assignment
- Fare Calculation
- Ride Status Tracking
- Ride History
- Logging
- Database Storage

The application does not include GPS integration, live maps, online payments, OTP verification, or push notifications.

1.5 Real-World Applications

The concepts implemented in this project are used in:

- Uber
- Ola
- Lyft
- Rapido
- Taxi Management Systems
- Fleet Management Platforms
- Logistics Applications

---

2. Problem Statement

Traditional taxi booking systems require manual coordination between passengers and drivers, leading to delays, booking errors, and inefficient ride management.

The **Ride Booking Backend** automates the complete ride lifecycle by managing passengers, drivers, ride requests, assignments, fare calculations, and ride completion while maintaining accurate records in a database.

Expected Users

- Ride-sharing Companies
- Taxi Operators
- Fleet Managers
- Java Backend Developers

- Software Engineering Students

---

3. Features

3.1 Passenger Management

Manage passenger information including:

- Passenger ID
- Name
- Phone Number
- Email Address

3.2 Driver Management

Manage driver details including:

- Driver ID
- Name
- Phone Number
- License Number
- Driver Availability

3.3 Vehicle Management

Maintain vehicle information such as:

- Vehicle ID
- Vehicle Number
- Vehicle Type
- Registration Details

3.4 Ride Booking

Passengers can request rides by providing:

- Pickup Location
- Destination
- Ride Type (Optional)

The system generates a unique ride request.

3.5 Driver Assignment

The system assigns an available driver to a ride request and updates driver availability accordingly.

3.6 Fare Calculation

Calculate the ride fare based on predefined business rules such as:

- Base Fare
- Distance Charges
- Additional Charges (Optional)

3.7 Ride Status Tracking

Track rides through different stages:

- Requested
- Driver Assigned
- Driver Arrived
- Ride Started
- Ride Completed
- Ride Cancelled

3.8 Ride History

Display ride details including:

- Ride ID
- Passenger

- Driver
- Pickup Location
- Destination
- Fare
- Ride Status

3.9 Logging

Record:

- Ride Requests
- Driver Assignments
- Ride Completion
- Fare Calculations
- System Errors

3.10 Unit Testing

Test:

- Ride Booking
- Driver Assignment
- Fare Calculation
- Ride Status Updates
- Business Rules

---

4. Functional Requirements

The system shall:

FR-01 Register passengers.

FR-02 Register drivers.

FR-03 Manage vehicle information.

FR-04 Accept ride requests.

FR-05 Assign drivers to rides.

FR-06 Calculate ride fares.

FR-07 Update ride status.

FR-08 Display ride history.

FR-09 Store ride information in MySQL.

FR-10 Record application logs.

FR-11 Support automated unit testing.

FR-12 Handle runtime and database exceptions.

---

5. Non-Functional Requirements

Performance

Ride requests and driver assignments should be processed efficiently.

Reliability

Ride records, driver availability, and fare calculations should remain accurate.

Maintainability

The application should follow Clean Architecture and SOLID principles.

Scalability

The architecture should support future integration with GPS services, payment gateways, REST APIs, and mobile applications.

Security

The application should validate all input data before processing.

Logging

Important system events and errors should be recorded for monitoring and debugging.

Testability

Business logic should be independently testable using automated unit tests.

---

6. Assumptions and Constraints

Assumptions

- Every passenger has a unique Passenger ID.
- Every driver has a unique Driver ID.
- Every vehicle has a unique Vehicle ID.
- Every ride has a unique Ride ID.
- Only available drivers can be assigned to new rides.
- MySQL and Maven/Gradle are properly configured.

Constraints

- Java Backend Application
- Java JDK 21 or later
- MySQL Database
- Maven or Gradle
- Clean Architecture
- SOLID Principles

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Represent passengers, drivers, vehicles, rides, and fare details as separate classes.
SOLID Principles	Design loosely coupled and maintainable backend components.
Design Patterns	Apply Factory, Singleton, Strategy, DAO, Builder, and Observer patterns where appropriate.
Clean Architecture	Separate business logic, persistence, and infrastructure layers.
Maven / Gradle	Manage project dependencies and builds.
JDBC / MySQL	Store passengers, drivers, vehicles, and ride information.
Collections & Generics	Manage ride requests and entity collections efficiently.
Exception Handling	Handle booking failures, invalid input, and database errors.
Logging	Record ride operations, assignments, and application events.
Unit Testing	Verify booking logic, fare calculation, and driver assignment.
Modular Programming	Organize the application into reusable modules and packages.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop enterprise-level transportation management software.
- Design scalable booking workflows.
- Apply SOLID principles and Design Patterns in complex applications.
- Build backend systems using Clean Architecture.
- Integrate Java with MySQL through JDBC.
- Implement business rules for fare calculation and ride assignment.
- Write maintainable and testable Java code.
- Use logging for monitoring and debugging.
- Understand ride lifecycle management.
- Build a strong foundation for backend systems used in transportation and logistics platforms.

Conclusion

The **Ride Booking Backend (Uber Mini)** is a professional-level Java backend project that simulates the core functionality of modern ride-hailing platforms. By implementing passenger management, driver assignment, vehicle management, ride booking, fare calculation, ride

tracking, and persistent database storage, learners gain practical experience in designing scalable enterprise applications. This project provides an excellent foundation for developing transportation platforms, logistics systems, fleet management software, and distributed backend services.

Software Requirements Specification (SRS)

Project Name

Banking Backend (Transaction Engine)

Project Type: Java Backend Application
Difficulty Level: ⭑ ⭑ ⭑ ⭑ ⭑ Professional
Programming Language: Java
Build Tool: Maven / Gradle
Database: MySQL
Connectivity: JDBC (Future Enhancement: JPA/Hibernate)
Architecture: Clean Architecture

---

1. Project Overview

1.1 Introduction

The **Banking Backend (Transaction Engine)** is a professional Java backend application that simulates the core transaction processing system of a banking platform. The application manages customers, bank accounts, deposits, withdrawals, fund transfers, account balances, and transaction history while ensuring accurate and secure financial operations.

Unlike simple banking applications, this project focuses on the **backend transaction engine**, where every financial operation follows strict business rules to maintain data consistency and integrity. It demonstrates enterprise software engineering using **SOLID Principles**, **Design Patterns**, **Clean Architecture**, **Maven/Gradle**, **Logging**, and **Unit Testing**.

This project serves as the capstone project of the Professional phase and prepares learners for enterprise banking and financial software development.

1.2 Purpose

The purpose of this project is to build a reliable banking transaction engine capable of processing financial transactions securely while exposing learners to enterprise backend architecture and business logic implementation.

1.3 Objectives

The objectives of this project are to:

- Manage customer accounts.
- Process deposits and withdrawals.
- Transfer funds between accounts.
- Maintain transaction history.
- Validate banking business rules.
- Store financial data securely.
- Apply professional software engineering principles.

1.4 Scope

The Banking Backend manages core banking operations.

The system includes:

- Customer Management
- Account Management
- Deposit Processing
- Withdrawal Processing
- Fund Transfer
- Transaction History
- Account Balance Management
- Logging
- Database Storage

The application does not include ATM integration, internet banking authentication, loan processing, credit cards, interest calculation, or mobile banking applications.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- Core Banking Systems
- Internet Banking Platforms
- Digital Banking Applications
- Financial Transaction Systems
- Payment Processing Platforms
- Enterprise Banking Software

---

2. Problem Statement

Banks process thousands of financial transactions every day. Manual transaction management is slow, error-prone, and unsuitable for maintaining financial accuracy. Incorrect transaction processing can lead to inconsistent account balances and data integrity issues.

The **Banking Backend** automates financial operations by securely managing customer accounts, validating transactions, processing deposits, withdrawals, and fund transfers, while maintaining accurate transaction records.

Expected Users

- Banks
- Financial Institutions
- FinTech Companies
- Java Backend Developers
- Software Engineering Students

---

3. Features

3.1 Customer Management

Manage customer information including:

- Customer ID
- Name
- Email Address
- Phone Number

- Address

3.2 Account Management

Create and manage bank accounts including:

- Account Number
- Account Type
- Current Balance
- Account Status

3.3 Deposit Money

Customers can deposit money into their accounts.

The system automatically updates the account balance and records the transaction.

3.4 Withdraw Money

Customers can withdraw money from their accounts.

The system verifies sufficient balance before processing the withdrawal.

3.5 Fund Transfer

Transfer money between two bank accounts.

The system:

- Validates both accounts.
- Checks available balance.
- Deducts money from the sender.
- Credits the receiver.
- Records both transactions.

3.6 Transaction History

Maintain complete transaction records including:

- Transaction ID
- Transaction Type
- Amount
- Date and Time
- Source Account
- Destination Account (for transfers)
- Transaction Status

3.7 Balance Inquiry

Display:

- Account Number
- Account Holder
- Current Balance
- Recent Transactions

3.8 Logging

Record:

- Account creation
- Deposits
- Withdrawals
- Fund transfers
- Failed transactions
- System errors

3.9 Unit Testing

Test:

- Deposit logic
- Withdrawal validation

- Fund transfer logic
- Balance calculations
- Business rules

---

4. Functional Requirements

The system shall:

FR-01 Register customers.

FR-02 Create bank accounts.

FR-03 Process deposits.

FR-04 Process withdrawals.

FR-05 Transfer funds between accounts.

FR-06 Display account balances.

FR-07 Maintain transaction history.

FR-08 Store banking records in MySQL.

FR-09 Record application logs.

FR-10 Support automated unit testing.

FR-11 Validate banking business rules.

FR-12 Handle runtime and database exceptions.

---

5. Non-Functional Requirements

Performance

The application should process financial transactions efficiently with minimal response time.

Reliability

The system should ensure accurate account balances and transaction records after every operation.

Maintainability

The application should follow Clean Architecture and SOLID principles to support future enhancements.

Scalability

The architecture should support future integration with REST APIs, payment gateways, authentication systems, and distributed microservices.

Security

The application should validate all transactions, prevent invalid operations, and protect sensitive banking data.

Logging

All financial transactions, errors, and system activities should be logged for auditing and debugging purposes.

Testability

All business logic should be independently testable through automated unit tests.

---

6. Assumptions and Constraints

Assumptions

- Every customer has a unique Customer ID.
- Every bank account has a unique Account Number.
- Every transaction has a unique Transaction ID.
- Withdrawals and transfers are allowed only when sufficient funds are available.
- Negative account balances are not permitted.
- MySQL Server and Maven/Gradle are properly configured.

Constraints

- Java Backend Application
- Java JDK 21 or later
- MySQL Database
- Maven or Gradle
- Clean Architecture
- SOLID Principles

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Represent customers, accounts, and transactions as separate classes.
SOLID Principles	Design maintainable, extensible, and loosely coupled banking services.
Design Patterns	Apply Factory, Singleton, Strategy, DAO, Service Layer, and Builder patterns where appropriate.
Clean Architecture	Separate business rules, persistence, and infrastructure layers.
Maven / Gradle	Manage project dependencies, plugins, and builds.
JDBC / MySQL	Store customer, account, and transaction information permanently.
Collections & Generics	Manage banking entities and transaction records efficiently.
Exception Handling	Handle invalid transactions, insufficient balance, and database failures.
Logging	Record banking operations for auditing and troubleshooting.
Unit Testing	Verify transaction processing, balance calculations, and business rules.
Modular Programming	Organize the application into scalable packages and reusable modules.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Develop enterprise-grade financial backend applications.
- Implement secure transaction processing logic.
- Apply SOLID principles and Design Patterns to large-scale software.
- Build scalable backend systems using Clean Architecture.
- Integrate Java with MySQL using JDBC.
- Validate complex business rules and financial workflows.
- Write modular, maintainable, and testable code.
- Implement structured logging for auditing and debugging.
- Understand transaction lifecycle management in banking systems.
- Build a strong foundation for enterprise Java development using Spring Boot, Hibernate, Microservices, and cloud-native backend applications.

Conclusion

The **Banking Backend (Transaction Engine)** is the capstone project of the Professional phase and represents a complete enterprise-level backend system. By implementing customer management, account management, deposits, withdrawals, fund transfers, transaction history, logging, and persistent database storage, learners gain hands-on experience in designing reliable, scalable, and maintainable financial software. This project consolidates all the concepts learned throughout the roadmap—including Object-Oriented Programming, JDBC, MySQL, SOLID Principles, Design Patterns, Clean Architecture, Maven/Gradle, Logging, and Unit Testing—and provides an excellent foundation for building production-grade banking systems, payment platforms, FinTech applications, and enterprise backend services.

Software Requirements Specification (SRS)

Project Name

Distributed Chat Server

Project Type: Java Distributed Server Application
Difficulty Level: ⭑ ⭑ ⭑ ⭑ ⭑ Advanced Professional
Programming Language: Java
Build Tool: Maven / Gradle
Architecture: Client-Server Distributed Architecture

---

1. Project Overview

1.1 Introduction

The **Distributed Chat Server** is a Java-based distributed networking application that enables multiple clients to communicate with each other through one or more interconnected chat servers. Unlike a basic chat application, this project focuses on handling concurrent client connections, synchronizing shared resources, and managing communication across distributed server instances.

The application introduces advanced concepts such as **Multithreading**, **Socket Networking**, **Thread Synchronization**, **Concurrent Programming**, and **Server Architecture**, preparing learners for real-world distributed systems development.

1.2 Purpose

The purpose of this project is to build a scalable and reliable chat server capable of handling multiple simultaneous client connections while maintaining synchronized communication between connected users.

1.3 Objectives

The objectives of this project are to:

- Accept multiple client connections simultaneously.
- Exchange messages between connected users.
- Synchronize access to shared resources.
- Maintain active client sessions.
- Broadcast messages efficiently.
- Build a distributed server architecture.
- Practice concurrent programming and networking.

1.4 Scope

The Distributed Chat Server provides real-time communication services.

The system includes:

- Server Management
- Client Connection Management
- User Registration
- Message Broadcasting
- Private Messaging (Optional)
- Thread Synchronization
- Client Session Management
- Distributed Communication
- Logging

The project does not include video calls, voice communication, media sharing, or end-to-end encryption.

1.5 Real-World Applications

The concepts implemented in this project are used in:

- Discord
- Slack
- Microsoft Teams
- WhatsApp Backend
- Telegram Servers
- Multiplayer Game Servers
- Enterprise Messaging Systems

---

2. Problem Statement

Real-time communication systems must support hundreds or thousands of users simultaneously. When multiple clients send messages at the same time, race conditions and inconsistent shared data can occur if concurrent access is not properly managed.

The **Distributed Chat Server** solves this problem by combining networking, multithreading, and synchronization mechanisms to ensure that messages are delivered correctly while multiple users communicate concurrently.

Expected Users

- Java Backend Developers
- Networking Students
- Distributed Systems Learners
- Software Engineering Students

---

3. Features

3.1 Server Management

- Start and stop the chat server.
- Listen for incoming client connections.
- Monitor connected clients.

3.2 Client Connection

Clients can connect using:

- Server IP Address
- Port Number
- Username

The server validates each connection.

3.3 User Registration

Each connected client registers with a unique username before joining the chat session.

3.4 Real-Time Messaging

Connected users can:

- Send text messages
- Receive messages instantly
- Broadcast messages to all connected users

3.5 Client Session Management

The server maintains:

- Active client list
- Connection status
- Login time
- Disconnection events

3.6 Synchronization

Shared resources such as:

- Connected client list
- Message queue
- Server logs

must be synchronized to prevent race conditions.

3.7 Distributed Communication

Support communication between multiple server instances (basic distributed architecture).

3.8 Logging

Maintain logs for:

- Client connections
- Client disconnections
- Message delivery
- Server errors
- Synchronization events

---

4. Functional Requirements

The system shall:

FR-01 Start the chat server.

FR-02 Accept multiple client connections.

FR-03 Register connected users.

FR-04 Exchange messages between clients.

FR-05 Broadcast messages.

FR-06 Maintain active client sessions.

FR-07 Synchronize shared resources.

FR-08 Record server logs.

FR-09 Detect disconnected clients.

FR-10 Handle concurrent communication safely.

FR-11 Validate incoming connections.

FR-12 Handle networking exceptions gracefully.

---

5. Non-Functional Requirements

Performance

The server should support multiple simultaneous users with minimal communication delay.

Reliability

Messages should be delivered accurately without duplication or loss.

Scalability

The architecture should support additional server instances and increasing client connections.

Availability

The server should remain operational while handling continuous client requests.

Maintainability

The application should follow modular design principles for future enhancements.

Security

The server should validate user input and reject invalid connection requests.

Thread Safety

Shared resources must remain consistent during concurrent access.

---

6. Assumptions and Constraints

Assumptions

- Every client has a unique username.
- The server starts before client connections.
- Communication uses TCP sockets.
- Network connectivity is available.

Constraints

- Java JDK 21 or later
- Java Socket Programming
- TCP Protocol
- Maven or Gradle
- Console-based Server
- No database
- No encryption

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Represent clients, servers, and messages as objects.
Socket Programming	Enable communication between clients and servers.
Networking	Exchange messages across systems connected through TCP/IP.
Multithreading	Handle multiple client connections simultaneously.
Synchronization	Protect shared resources from concurrent modification.
Concurrent Collections	Manage connected clients safely in multithreaded environments.
Exception Handling	Handle socket failures and communication errors.
Collections & Generics	Store client sessions and message queues.
Logging	Record server activities and communication events.
Maven / Gradle	Build and manage project dependencies.
Modular Programming	Organize networking components into reusable modules.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Build real-time distributed networking applications.
- Understand client-server architecture.
- Develop multithreaded server applications.
- Apply synchronization techniques to prevent race conditions.
- Manage concurrent client connections efficiently.
- Design scalable communication systems.
- Handle network exceptions professionally.
- Develop enterprise-style server software.
- Strengthen knowledge of distributed systems.
- Build a strong foundation for cloud computing, distributed computing, and microservices.

Conclusion

The **Distributed Chat Server** is an advanced professional Java project that introduces learners to concurrent network programming and distributed system design. By implementing multithreaded client handling, synchronized shared resources, real-time message broadcasting, and distributed server communication, learners gain practical experience in building scalable

server-side applications. This project serves as a strong foundation for enterprise messaging platforms, multiplayer game servers, collaborative applications, cloud-based communication systems, and distributed backend architectures.

Software Requirements Specification (SRS)

Project Name

Tiny Java Database Engine

Project Type: Java Database Engine (Educational DBMS)
Difficulty Level: ⭑ ⭑ ⭑ ⭑ ⭑ ⭑ Expert
Programming Language: Java
Build Tool: Maven / Gradle
Architecture: Layered Architecture

---

1. Project Overview

1.1 Introduction

The **Tiny Java Database Engine** is an educational database management system (DBMS) developed entirely in Java. Instead of using an existing database such as MySQL or PostgreSQL, this project builds the core components of a database engine from scratch.

The system allows users to create tables, insert records, store data on disk, build indexes, parse SQL-like queries, search records efficiently, update data, and delete records. Through this project, learners gain a deep understanding of how modern relational databases work internally.

Rather than being a full-featured commercial database, this project focuses on implementing the fundamental building blocks of a DBMS in a simplified and educational manner.

1.2 Purpose

The purpose of this project is to understand the internal architecture of database systems by implementing storage management, indexing, query processing, and record management from scratch using Java.

1.3 Objectives

The objectives of this project are to:

- Build a lightweight database engine.
- Create and manage tables.
- Store records permanently.
- Implement indexing for faster searches.
- Parse SQL-like commands.
- Execute database operations.
- Understand internal DBMS architecture.

1.4 Scope

The Tiny Java Database Engine provides a simplified relational database system.

The system includes:

- Database Creation
- Table Management
- Record Storage
- Record Retrieval
- Record Update
- Record Deletion
- Index Management
- Query Parser
- Storage Engine
- Logging

The application does not include distributed databases, transactions, query optimization, replication, or multi-user concurrency control.

1.5 Real-World Applications

The concepts implemented in this project are fundamental to:

- MySQL

- PostgreSQL
- Oracle Database
- Microsoft SQL Server
- SQLite
- MariaDB
- Apache Derby
- Embedded Database Systems

---

2. Problem Statement

Applications require efficient mechanisms to store, retrieve, update, and manage large volumes of structured data. While existing database systems provide these capabilities, developers often use them without understanding their internal implementation.

The **Tiny Java Database Engine** addresses this learning gap by implementing the core components of a relational database from scratch, allowing learners to understand how tables, indexes, storage engines, and query processors operate internally.

Expected Users

- Database Students
- Java Developers
- Software Engineering Students
- DBMS Learners
- Backend Developers

---

3. Features

3.1 Database Management

Users can:

- Create databases
- Open databases
- Delete databases

3.2 Table Management

Users can:

- Create tables
- Delete tables
- View table schema
- List available tables

Each table contains:

- Table Name
- Columns
- Data Types
- Primary Key (Basic)

3.3 Record Management

Support basic CRUD operations:

- Insert Records
- View Records
- Update Records
- Delete Records

3.4 Storage Engine

The system stores table data in binary or text files.

Responsibilities include:

- Reading records
- Writing records
- Updating records
- Managing storage files

3.5 Index Management

Create indexes on selected columns to improve search performance.

The system maintains index structures separately from table data.

3.6 Query Parser

Support simplified SQL-like commands such as:

- CREATE TABLE
- INSERT INTO
- SELECT
- UPDATE
- DELETE
- DROP TABLE

The parser validates syntax and executes corresponding operations.

3.7 Search Engine

Retrieve records using:

- Primary Key
- Indexed Columns
- Sequential Scan (when no index exists)

3.8 Logging

Record:

- Database creation
- Table creation
- Query execution
- Storage operations
- Errors

---

4. Functional Requirements

The system shall:

FR-01 Create databases.

FR-02 Create tables.

FR-03 Insert records.

FR-04 Retrieve records.

FR-05 Update records.

FR-06 Delete records.

FR-07 Create indexes.

FR-08 Parse SQL-like commands.

FR-09 Store data permanently.

FR-10 Record database logs.

FR-11 Validate query syntax.

FR-12 Handle runtime exceptions.

---

5. Non-Functional Requirements

Performance

Indexed searches should execute faster than sequential scans.

Reliability

The storage engine should preserve data integrity during read and write operations.

Maintainability

The database engine should be modular so that future components such as transactions and query optimization can be added.

Scalability

The architecture should support future enhancements including B+ Trees, caching, transactions, and concurrency control.

Security

The system should validate SQL-like commands before execution.

Logging

All database operations and system errors should be recorded for debugging and analysis.

Testability

Each database module should be independently testable through unit tests.

---

6. Assumptions and Constraints

Assumptions

- Every table has a unique name.
- Column names are unique within a table.
- Records follow the table schema.
- Storage files remain accessible.
- Queries follow the supported SQL syntax.

Constraints

- Java JDK 21 or later
- Maven or Gradle
- Local File Storage

- Single-user database
- Educational implementation
- No transaction support
- No distributed storage

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Model databases, tables, records, indexes, and queries as objects.
File Handling	Store table data, metadata, and indexes on disk.
Collections & Generics	Manage records, schemas, indexes, and query results efficiently.
Exception Handling	Handle invalid queries, storage failures, and parsing errors.
Design Patterns	Apply Factory, DAO, Builder, Strategy, and Singleton patterns where appropriate.
SOLID Principles	Design maintainable and extensible database components.
Clean Architecture	Separate query processing, storage engine, indexing, and metadata management into independent layers.
String Processing	Tokenize and parse SQL-like commands.
Logging	Record query execution, storage operations, and errors.
Unit Testing	Verify parser logic, storage engine, indexing, and CRUD operations.
Modular Programming	Organize the database engine into reusable components.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Understand the internal architecture of relational database systems.
- Design and implement a basic storage engine.
- Build a SQL-like query parser.
- Implement table and record management.
- Create and use indexes for faster searching.
- Develop modular backend systems using professional software engineering principles.
- Apply SOLID Principles and Design Patterns in systems programming.
- Strengthen knowledge of file organization and data storage.
- Gain practical insight into how commercial database systems operate internally.
- Build a strong foundation for studying advanced DBMS topics such as B+ Trees, transaction management, query optimization, concurrency control, recovery mechanisms, and distributed databases.

Conclusion

The **Tiny Java Database Engine** is an expert-level systems programming project that introduces learners to the internal workings of relational database management systems. By implementing table management, record storage, indexing, SQL-like query parsing, storage engines, and persistent data management, learners move beyond simply using databases to understanding how they are built. This project serves as an exceptional bridge toward advanced database engineering, storage engine development, distributed databases, and large-scale backend infrastructure.

Software Requirements Specification (SRS)

Project Name

Mini Java Compiler

Project Type: Java Compiler (Educational Compiler)
Difficulty Level: ⭐⭐⭐⭐⭐⭐ Expert
Programming Language: Java
Build Tool: Maven / Gradle
Architecture: Compiler Pipeline Architecture

---

1. Project Overview

1.1 Introduction

The **Mini Java Compiler** is an educational compiler developed entirely in Java to demonstrate how programming languages are translated into machine-understandable representations. Rather than executing Java programs directly, this project implements the fundamental stages of a compiler, including **Lexical Analysis**, **Syntax Analysis**, **Abstract Syntax Tree (AST) Construction**, and **Semantic Analysis**.

Unlike using existing compilers such as `javac`, this project allows learners to build a simplified compiler from scratch and understand how source code is analyzed before execution.

The compiler will support a small subset of Java syntax for educational purposes, making it easier to study compiler internals without the complexity of the full Java language.

1.2 Purpose

The purpose of this project is to understand compiler design by implementing the core phases involved in translating source code into an internal representation.

1.3 Objectives

The objectives of this project are to:

- Read Java source code.
- Perform lexical analysis.
- Generate tokens.
- Parse program syntax.
- Construct an Abstract Syntax Tree (AST).
- Perform semantic analysis.
- Detect syntax and semantic errors.
- Understand compiler architecture.

1.4 Scope

The Mini Java Compiler provides a simplified compiler pipeline.

The system includes:

- Source Code Reader
- Lexer
- Token Generator
- Parser
- AST Builder
- Symbol Table
- Semantic Analyzer
- Error Reporting
- Logging

The project does not include code optimization, bytecode generation, machine code generation, JVM implementation, or full Java language support.

1.5 Real-World Applications

The concepts implemented in this project are fundamental to:

- Java Compiler (javac)
- GCC
- Clang
- Kotlin Compiler
- Scala Compiler
- C# Compiler (Roslyn)
- Language Interpreters
- IDE Code Analysis Tools

---

2. Problem Statement

Programming languages cannot be executed directly by computers. Source code must first be analyzed, validated, and transformed into an internal representation before it can be executed or translated further.

The **Mini Java Compiler** solves this problem by implementing the essential phases of compiler design, allowing learners to understand how programming languages are processed internally.

Expected Users

- Compiler Design Students
- Java Developers
- Software Engineering Students
- Programming Language Enthusiasts
- Backend and Systems Developers

---

3. Features

3.1 Source Code Reader

Read Java source code from text files.

Supported constructs may include:

- Variable declarations
- Assignments
- Arithmetic expressions
- Conditional statements
- Loops
- Method declarations (basic)

3.2 Lexical Analysis (Lexer)

The lexer scans source code and converts characters into tokens.

Supported token types include:

- Keywords
- Identifiers
- Operators
- Literals
- Delimiters
- Symbols

3.3 Token Generation

Generate a sequence of tokens representing the input program.

Example:

```
int a = 10;
```

Produces tokens such as:

- KEYWORD
- IDENTIFIER
- ASSIGNMENT_OPERATOR
- INTEGER_LITERAL
- SEMICOLON

3.4 Syntax Analysis (Parser)

The parser validates whether the token sequence follows the language grammar.

The parser constructs a parse tree or directly builds an Abstract Syntax Tree.

3.5 Abstract Syntax Tree (AST)

Generate an AST representing the logical structure of the source program.

Each node represents constructs such as:

- Program
- Variable Declaration
- Assignment
- Expression
- If Statement
- While Loop
- Method Declaration

3.6 Symbol Table

Maintain information about:

- Variables
- Data Types
- Scope
- Methods

The symbol table supports semantic analysis.

3.7 Semantic Analysis

Perform semantic validation including:

- Undeclared Variables
- Duplicate Variable Declarations
- Type Compatibility

- Scope Checking
- Assignment Validation
- Method Validation (Basic)

3.8 Error Reporting

Display meaningful compiler errors including:

- Lexical Errors
- Syntax Errors
- Semantic Errors
- Line Numbers
- Error Descriptions

3.9 Logging

Record:

- Compilation Process
- Lexer Output
- Parser Activities
- Semantic Analysis Results
- Compiler Errors

---

4. Functional Requirements

The system shall:

FR-01 Read Java source files.

FR-02 Perform lexical analysis.

FR-03 Generate tokens.

FR-04 Parse program syntax.

FR-05 Construct an Abstract Syntax Tree.

FR-06 Maintain a symbol table.

FR-07 Perform semantic analysis.

FR-08 Detect compilation errors.

FR-09 Log compiler operations.

FR-10 Validate supported Java syntax.

FR-11 Handle compilation exceptions.

FR-12 Produce structured compiler output.

---

5. Non-Functional Requirements

Performance

The compiler should process source files efficiently.

Reliability

The compiler should consistently detect lexical, syntax, and semantic errors.

Maintainability

Each compiler phase should be implemented independently to simplify future enhancements.

Scalability

The architecture should support future additions such as intermediate code generation, optimization, and bytecode generation.

Security

Only valid source files should be accepted for compilation.

Logging

All compiler stages and detected errors should be logged for debugging purposes.

Testability

Each compiler component should be independently testable using unit tests.

---

6. Assumptions and Constraints

Assumptions

- Source files follow the supported Java subset.
- Only supported grammar constructs are accepted.
- Input files are encoded in UTF-8.
- Compilation is performed locally.

Constraints

- Java JDK 21 or later
- Maven or Gradle
- Educational Compiler
- Supports a simplified Java grammar
- No JVM bytecode generation
- No machine code generation

---

7. Java Skills Required

Java Concept	Application in the Project
Object-Oriented Programming	Represent tokens, AST nodes, symbol tables, and compiler phases as classes.
String Processing	Read and tokenize source code efficiently.
Collections & Generics	Store tokens, AST nodes, grammar rules, and symbol tables.
File Handling	Read source code files for compilation.
Exception Handling	Handle invalid syntax, lexical errors, and semantic violations.
Design Patterns	Apply Factory, Visitor, Builder, Strategy, and Singleton patterns where appropriate.
SOLID Principles	Design modular compiler components.
Clean Architecture	Separate lexical analysis, parsing, semantic analysis, and utilities into independent modules.
Logging	Record compiler execution and error diagnostics.
Unit Testing	Verify lexer, parser, AST generation, and semantic analysis independently.
Modular Programming	Organize compiler phases into reusable packages.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Understand every major phase of compiler design.
- Implement a lexical analyzer from scratch.
- Build recursive-descent or similar parsers.
- Construct and traverse Abstract Syntax Trees.
- Design and use symbol tables effectively.
- Perform semantic analysis and type checking.
- Detect and report compiler errors professionally.
- Apply software engineering principles to systems programming.
- Gain a strong understanding of programming language implementation.
- Build a solid foundation for advanced compiler topics such as intermediate code generation, optimization, control-flow analysis, data-flow analysis, JVM bytecode generation, and language development.

Conclusion

The **Mini Java Compiler** is an expert-level systems programming project that introduces learners to the internal architecture of modern compilers. By implementing lexical analysis, syntax analysis, Abstract Syntax Tree generation, symbol table management, semantic analysis,

and compiler diagnostics, learners gain hands-on experience in one of the most fundamental areas of computer science. This project provides a strong foundation for advanced compiler construction, programming language development, static code analysis tools, integrated development environments (IDEs), and virtual machine technologies.

Software Requirements Specification (SRS)

Project Name

Distributed Microservices E-Commerce Platform

Project Type: Enterprise Distributed Microservices Platform
Difficulty Level: ⋆ ⋆ ⋆ ⋆ ⋆ ⋆ ⋆ Elite / Enterprise Level
Programming Language: Java 21+
Framework: Spring Boot
Architecture: Microservices Architecture
Database: PostgreSQL
Message Broker: Apache Kafka
Cache: Redis
Containerization: Docker
Orchestration: Kubernetes
Search Engine: Elasticsearch

---

1. Project Overview

1.1 Introduction

The **Distributed Microservices E-Commerce Platform** is a large-scale enterprise backend application that simulates the architecture of modern e-commerce systems such as **Amazon**, **Flipkart**, **Shopify**, and **eBay**.

Unlike a traditional monolithic application, this project is built using **Microservices Architecture**, where each business capability is implemented as an independent service. Every service has its own database, communicates with other services through REST APIs and asynchronous messaging using **Apache Kafka**, caches frequently accessed data using **Redis**, and is deployed independently using **Docker** and **Kubernetes**.

This project represents a production-style backend system capable of handling scalability, fault isolation, distributed communication, and cloud-native deployment.

1.2 Purpose

The purpose of this project is to design and implement a cloud-native distributed e-commerce backend that demonstrates enterprise software architecture, distributed systems principles, and modern DevOps practices.

1.3 Objectives

The objectives of this project are to:

- Develop independent microservices.
- Implement distributed communication.
- Manage products and inventory.
- Process customer orders.
- Handle asynchronous events.
- Improve performance using caching.
- Support scalable deployments.
- Apply enterprise architecture patterns.

1.4 Scope

The platform provides backend services for an e-commerce application.

The system includes:

- User Service
- Authentication Service
- Product Service
- Inventory Service
- Cart Service
- Order Service
- Payment Service (Simulation)
- Notification Service
- Search Service
- API Gateway
- Service Discovery
- Distributed Logging

- Monitoring
- Containerized Deployment

The project excludes real payment gateway integration, recommendation engines, machine learning models, and third-party shipping providers.

1.5 Real-World Applications

The concepts implemented in this project are used in:

- Amazon
- Flipkart
- Shopify
- eBay
- Walmart
- Netflix (Microservices Architecture)
- Uber Backend
- Enterprise Cloud Platforms

---

2. Problem Statement

As online businesses grow, monolithic applications become difficult to maintain, scale, and deploy. A failure in one module may affect the entire application, and scaling individual components independently becomes impossible.

The **Distributed Microservices E-Commerce Platform** addresses these challenges by decomposing the application into independent services that communicate efficiently while remaining loosely coupled. This architecture improves scalability, maintainability, availability, and deployment flexibility.

Expected Users

- Enterprise Software Developers
- Backend Engineers
- Cloud Engineers
- DevOps Engineers
- Java Professionals
- Software Architects

---

3. Features

3.1 User Service

Manage:

- Customer Registration
- User Profiles
- User Information

3.2 Authentication Service

Provide:

- User Login
- Password Verification
- Token Generation (Future Enhancement)
- Authorization

3.3 Product Service

Manage:

- Products
- Categories
- Product Details
- Product Availability

3.4 Inventory Service

Maintain:

- Stock Levels
- Inventory Updates
- Low Stock Alerts
- Product Availability

3.5 Cart Service

Allow customers to:

- Add Products
- Remove Products
- Update Quantities
- View Shopping Cart

3.6 Order Service

Handle:

- Order Creation
- Order Validation
- Order Status
- Order History

3.7 Payment Service (Simulation)

Simulate:

- Payment Requests
- Payment Success
- Payment Failure
- Transaction Status

3.8 Notification Service

Generate notifications for:

- Order Confirmation
- Payment Status
- Shipping Updates

3.9 Search Service

Provide fast product searching using:

- Product Name
- Category
- Keywords
- Elasticsearch Indexes

3.10 Kafka Event Processing

Exchange asynchronous events including:

- Order Created
- Payment Completed
- Inventory Updated
- Notification Triggered

3.11 Redis Cache

Cache:

- Product Information
- Frequently Accessed Products
- User Sessions
- Search Results

3.12 Docker Deployment

Package each microservice as an independent Docker container.

3.13 Kubernetes Deployment

Deploy and manage microservices using Kubernetes.

Support:

- Scaling
- Self-Healing
- Load Balancing
- Rolling Updates

---

4. Functional Requirements

The system shall:

FR-01 Register and manage users.

FR-02 Authenticate users.

FR-03 Manage products.

FR-04 Maintain inventory.

FR-05 Manage shopping carts.

FR-06 Process customer orders.

FR-07 Simulate payment processing.

FR-08 Publish and consume Kafka events.

FR-09 Cache frequently accessed data using Redis.

FR-10 Perform product searches using Elasticsearch.

FR-11 Deploy services using Docker.

FR-12 Orchestrate services using Kubernetes.

---

5. Non-Functional Requirements

Performance

The platform should process thousands of requests with low latency.

Scalability

Individual microservices should scale independently according to workload.

Availability

Failure of one microservice should not interrupt the entire platform.

Reliability

The system should maintain data consistency across distributed services.

Maintainability

Each service should be independently developed, tested, and deployed.

Fault Tolerance

Temporary failures should not result in complete system downtime.

Security

The platform should validate all incoming requests and protect sensitive data.

Observability

Application logs and service health should be monitored continuously.

---

6. Assumptions and Constraints

Assumptions

- Every service owns its own database.
- Kafka Broker is available.
- Redis Server is operational.
- PostgreSQL is configured correctly.
- Docker Engine is installed.
- Kubernetes Cluster is available.
- Network connectivity exists between all services.

Constraints

- Java 21+
- Spring Boot
- PostgreSQL
- Apache Kafka
- Redis
- Docker
- Kubernetes
- Elasticsearch
- Microservices Architecture

---

7. Technologies & Skills Required

Technology / Concept	Application in the Project
Java	Core programming language for all microservices.
Spring Boot	Develop independent backend microservices.
Microservices Architecture	Split the application into independent deployable services.
REST APIs	Enable synchronous communication between services and clients.
Apache Kafka	Support asynchronous event-driven communication.
PostgreSQL	Store business data for each microservice.
Redis	Cache frequently accessed data to improve performance.
Elasticsearch	Implement fast and efficient product search functionality.
Docker	Containerize each microservice for consistent deployment.

Technology / Concept	Application in the Project
Kubernetes	Orchestrate containers, scaling, load balancing, and self-healing.
Design Patterns	Apply Factory, Strategy, Observer, Builder, Repository, and Dependency Injection patterns.
SOLID Principles	Design maintainable, extensible, and loosely coupled services.
Clean Architecture	Separate domain logic from infrastructure and presentation layers.
Logging & Monitoring	Record application events and monitor service health.
Unit & Integration Testing	Validate individual services and inter-service communication.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Design enterprise-scale distributed systems.
- Build production-ready Spring Boot microservices.
- Implement synchronous and asynchronous communication.
- Use Apache Kafka for event-driven architecture.
- Optimize performance with Redis caching.
- Design independent PostgreSQL databases for microservices.
- Implement full-text search using Elasticsearch.
- Containerize applications using Docker.
- Deploy and manage distributed applications with Kubernetes.
- Apply Clean Architecture, SOLID Principles, and Design Patterns in enterprise software.
- Understand cloud-native application development and distributed system design.
- Build a strong foundation for working as a Backend Engineer, Java Developer, Cloud Engineer, Platform Engineer, or Software Architect.

Conclusion

The **Distributed Microservices E-Commerce Platform** is an elite-level enterprise software project that integrates modern backend engineering, cloud-native architecture, distributed systems, and DevOps practices into a single production-style application. By combining Spring Boot, Kafka, Redis, PostgreSQL, Elasticsearch, Docker, and Kubernetes, learners gain hands-on experience building scalable, fault-tolerant, and highly maintainable backend systems similar to those used by leading technology companies. This project represents the culmination of the Java backend learning journey and prepares learners for professional roles in enterprise software development and cloud engineering.

Software Requirements Specification (SRS)

Project Name

Real-Time Collaborative Code Editor

Project Type: Enterprise Distributed Collaborative Development Platform
Difficulty Level: ⭑ ⭑ ⭑ ⭑ ⭑ ⭑ ⭑ ⭑ Master / Elite Level
Programming Language: Java
Framework: Spring Boot
Communication: WebSockets
Architecture: Microservices + Event-Driven Architecture
Database: PostgreSQL
Cache: Redis
Message Broker: Apache Kafka
Containerization: Docker
Orchestration: Kubernetes

---

1. Project Overview

1.1 Introduction

The **Real-Time Collaborative Code Editor** is a distributed software platform that enables multiple users to edit the same source code document simultaneously over the internet. Every modification made by one user is instantly synchronized with all connected collaborators while maintaining document consistency.

Unlike traditional text editors, this application solves one of the most challenging problems in distributed computing—**real-time synchronization of shared documents**. The system implements **Operational Transformation (OT)** or **Conflict-Free Replicated Data Types (CRDTs)** to resolve concurrent edits without data loss or conflicts.

The platform also supports secure authentication, collaborative editing, version history, and real-time communication using **WebSockets**.

This project closely resembles the backend architecture of collaborative development tools such as **Visual Studio Live Share**, **CodeSandbox**, **Replit**, and **Google Docs**.

1.2 Purpose

The purpose of this project is to develop a highly scalable collaborative editing platform that demonstrates advanced distributed systems concepts, real-time communication, synchronization algorithms, and enterprise backend architecture.

1.3 Objectives

The objectives of this project are to:

- Enable multiple users to edit the same document simultaneously.
- Synchronize document changes in real time.
- Resolve concurrent editing conflicts.
- Maintain complete version history.
- Authenticate users securely.
- Store documents reliably.
- Apply distributed systems principles.

1.4 Scope

The Real-Time Collaborative Code Editor provides collaborative coding services.

The system includes:

- User Authentication
- Workspace Management
- Document Management
- Live Code Editing
- Real-Time Synchronization
- Conflict Resolution
- Version History
- User Presence
- Logging
- Distributed Communication

The application does not include video conferencing, voice chat, integrated compiler execution, AI code completion, or deployment pipelines.

1.5 Real-World Applications

The concepts implemented in this project are used in:

- Google Docs
- Visual Studio Live Share
- Replit
- CodeSandbox
- GitHub Codespaces
- JetBrains Code With Me
- Figma (collaborative editing concepts)

---

2. Problem Statement

Modern software development often requires multiple developers to work on the same source code simultaneously. Without proper synchronization mechanisms, concurrent edits can overwrite each other, causing data loss and inconsistent document states.

The **Real-Time Collaborative Code Editor** solves this challenge by implementing distributed synchronization algorithms that guarantee every connected user eventually sees the same document while preserving all valid edits.

Expected Users

- Software Developers
- Development Teams
- Computer Science Students
- Backend Engineers
- Distributed Systems Learners

---

3. Features

3.1 User Authentication

Users can:

- Register
- Login
- Logout

- Manage user profiles

Authentication ensures only authorized users can access collaborative workspaces.

3.2 Workspace Management

Users can:

- Create coding workspaces
- Join shared workspaces
- Invite collaborators
- Manage project members

3.3 Code Document Management

Users can:

- Create files
- Open files
- Rename files
- Delete files
- Save files

3.4 Real-Time Collaborative Editing

Multiple users can edit the same file simultaneously.

Changes are synchronized instantly across all connected clients.

3.5 WebSocket Communication

The application uses WebSockets for:

- Instant document updates
- User presence notifications
- Typing indicators

- Cursor synchronization

3.6 Operational Transformation / CRDT

The synchronization engine:

- Detects concurrent edits
- Resolves editing conflicts
- Maintains document consistency
- Prevents lost updates

3.7 Version History

Maintain:

- Version Number
- Timestamp
- Author
- Change Summary

Users can restore previous document versions.

3.8 User Presence

Display:

- Online Users
- Active Editors
- Cursor Positions
- Currently Edited Files

3.9 Logging

Maintain logs for:

- User Login

- Document Updates
- Synchronization Events
- Version Creation
- System Errors

3.10 Testing

Verify:

- Synchronization correctness
- Conflict resolution
- WebSocket communication
- Authentication
- Version recovery

---

4. Functional Requirements

The system shall:

FR-01 Register and authenticate users.

FR-02 Create collaborative workspaces.

FR-03 Create and manage code files.

FR-04 Support simultaneous document editing.

FR-05 Synchronize edits using WebSockets.

FR-06 Resolve editing conflicts using OT or CRDT.

FR-07 Maintain document version history.

FR-08 Display active collaborators.

FR-09 Record application logs.

FR-10 Store project data in PostgreSQL.

FR-11 Validate user operations.

FR-12 Handle runtime and network exceptions.

---

5. Non-Functional Requirements

Performance

Document synchronization should occur with minimal latency.

Scalability

The platform should support thousands of concurrent collaborative sessions.

Reliability

No valid user edits should be lost during synchronization.

Availability

The collaboration service should remain available during continuous editing sessions.

Maintainability

The project should follow Clean Architecture and SOLID principles.

Security

Only authenticated users should access collaborative workspaces.

Fault Tolerance

Temporary network failures should not permanently lose document changes.

Consistency

All collaborators should eventually observe the same document state.

---

6. Assumptions and Constraints

Assumptions

- Every user has a unique account.
- Users have internet connectivity.
- WebSocket connections remain active during editing.
- PostgreSQL, Redis, Kafka, Docker, and Kubernetes are properly configured.

Constraints

- Java 21+
- Spring Boot
- WebSockets
- PostgreSQL
- Redis
- Apache Kafka
- Docker
- Kubernetes
- Operational Transformation (OT) or CRDT
- Clean Architecture

---

7. Technologies & Skills Required

Technology / Concept	Application in the Project
Java	Core backend development.

Technology / Concept	Application in the Project
Spring Boot	Build scalable backend services.
WebSockets	Enable bidirectional real-time communication.
Operational Transformation (OT)	Synchronize concurrent document edits.
CRDT	Alternative conflict-free synchronization mechanism.
PostgreSQL	Store users, workspaces, and document metadata.
Redis	Cache active sessions and frequently accessed document states.
Apache Kafka	Distribute editing events between services.
Docker	Containerize application services.
Kubernetes	Deploy and scale distributed services.
Design Patterns	Apply Observer, Strategy, Factory, Repository, Builder, and Dependency Injection patterns.
SOLID Principles	Develop maintainable and extensible software.
Clean Architecture	Separate synchronization logic, business rules, and infrastructure.
Logging	Record collaboration events and debugging information.
Unit & Integration Testing	Validate synchronization algorithms and distributed communication.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Build real-time collaborative applications.
- Develop distributed systems using WebSockets.
- Understand Operational Transformation and CRDT algorithms.
- Design scalable event-driven architectures.
- Implement concurrent editing synchronization.
- Build production-ready Spring Boot microservices.
- Apply SOLID Principles and Design Patterns in enterprise systems.
- Deploy distributed applications using Docker and Kubernetes.
- Design highly available collaborative platforms.
- Gain practical experience with advanced distributed computing concepts used in modern cloud applications.

Conclusion

The **Real-Time Collaborative Code Editor** is an elite-level distributed systems project that demonstrates one of the most challenging problems in modern software engineering: maintaining

consistency during concurrent edits by multiple users. By integrating WebSockets, Operational Transformation or CRDT, Spring Boot, Kafka, Redis, PostgreSQL, Docker, Kubernetes, and enterprise architectural principles, learners gain hands-on experience in designing scalable, fault-tolerant, and cloud-native collaboration platforms. This project serves as an excellent foundation for building professional collaborative development tools, cloud IDEs, distributed document editors, and next-generation real-time applications.

Software Requirements Specification (SRS)

Project Name

Developer Productivity Platform (Notion + GitHub + CI/CD)

Project Type: Enterprise Software Engineering Platform (Developer Productivity Suite)
Difficulty Level: ⋆ ⋆ ⋆ ⋆ ⋆ ⋆ ⋆ ⋆ ⋆ ⋆ Elite / Senior Software Engineer Level
Programming Language: Java 21+
Framework: Spring Boot
Architecture: Microservices + Clean Architecture + Event-Driven Architecture
Database: PostgreSQL
Cache: Redis
Message Broker: Apache Kafka
Search Engine: Elasticsearch
Containerization: Docker
Orchestration: Kubernetes

---

1. Project Overview

1.1 Introduction

The **Developer Productivity Platform** is an enterprise-grade software engineering platform designed to centralize project management, technical documentation, source code collaboration, code review workflows, analytics, and development automation into a unified system.

Inspired by platforms such as **Notion**, **GitHub**, **Jira**, **GitLab**, **Linear**, and modern DevOps tools, this platform enables development teams to plan projects, document software, manage repositories, review code, monitor engineering metrics, and extend functionality through a plugin architecture.

The application demonstrates enterprise software engineering concepts including **Microservices**, **Clean Architecture**, **SOLID Principles**, **Design Patterns**, **Distributed Systems**, **Plugin-Based Architecture**, **CI/CD Integration**, and optionally **AI-assisted development features**.

This project represents the culmination of the Java backend learning roadmap and closely resembles software built by senior engineers at large technology companies.

1.2 Purpose

The purpose of this project is to build a scalable, extensible, and cloud-native developer platform that improves software development productivity while demonstrating modern enterprise architecture and engineering practices.

1.3 Objectives

The objectives of this project are to:

- Manage software projects.
- Maintain technical documentation.
- Manage repositories and code reviews.
- Track development activities.
- Generate engineering analytics.
- Support plugins and extensions.
- Integrate with CI/CD pipelines.
- Demonstrate enterprise software architecture.

1.4 Scope

The Developer Productivity Platform provides backend services for software development teams.

The system includes:

- User Management
- Authentication & Authorization
- Workspace Management
- Project Management

- Task Management
- Documentation (Wiki)
- Repository Management
- Pull Request Management
- Code Review Workflow
- Analytics Dashboard
- Notification Service
- Plugin Framework
- CI/CD Integration
- Logging & Monitoring

Optional future modules include AI-assisted code review, documentation generation, sprint planning, and intelligent project insights.

1.5 Real-World Applications

The concepts implemented in this project are widely used in:

- GitHub
- GitLab
- Notion
- Jira
- Linear
- Azure DevOps
- Atlassian Confluence
- JetBrains Space

---

2. Problem Statement

Modern software teams use multiple disconnected tools for project management, documentation, version control, code reviews, analytics, and deployment. Switching between platforms reduces productivity and complicates collaboration.

The **Developer Productivity Platform** solves this problem by integrating essential software engineering workflows into a unified system that supports collaboration, extensibility, automation, and scalable enterprise development.

Expected Users

- Software Development Teams

- Enterprise Organizations
- Technical Leads
- Software Architects
- DevOps Engineers
- Java Backend Developers

---

3. Features

3.1 User Management

Manage users including:

- Registration
- Authentication
- User Profiles
- Roles
- Permissions

3.2 Workspace Management

Users can:

- Create Workspaces
- Invite Members
- Assign Roles
- Manage Teams

3.3 Project Management

Manage software projects including:

- Project Creation
- Milestones
- Sprint Planning
- Project Status
- Team Assignment

3.4 Documentation System

Create and manage:

- Technical Documentation
- Architecture Documents
- API Documentation
- Meeting Notes
- Project Wikis

Support version-controlled documentation.

3.5 Repository Management

Manage source code repositories including:

- Repository Information
- Branches
- Commit History
- Repository Members

3.6 Code Review Workflow

Support complete pull request lifecycle:

- Create Pull Requests
- Assign Reviewers
- Review Changes
- Approve Requests
- Reject Requests
- Merge Changes

3.7 Analytics Dashboard

Generate engineering metrics such as:

- Project Progress
- Code Review Statistics

- Pull Request Activity
- Team Productivity
- Repository Growth
- Sprint Completion Rate

3.8 Notification Service

Notify users about:

- Pull Requests
- Task Assignments
- Documentation Updates
- Project Changes
- Review Requests

3.9 Plugin Architecture

Allow developers to extend the platform by creating plugins.

Possible plugins include:

- Custom Reports
- Integrations
- Automation Rules
- Third-Party Services

3.10 CI/CD Integration

Integrate with build pipelines to display:

- Build Status
- Test Results
- Deployment Status
- Release History

3.11 AI-Assisted Features (Optional)

Provide optional intelligent features such as:

- Documentation Suggestions
- Code Review Assistance
- Task Recommendations
- Sprint Planning Suggestions
- Bug Analysis

3.12 Logging & Monitoring

Maintain logs for:

- User Activities
- Repository Events
- Project Updates
- Plugin Execution
- System Errors

---

4. Functional Requirements

The system shall:

FR-01 Register and authenticate users.

FR-02 Create and manage workspaces.

FR-03 Manage software projects.

FR-04 Maintain technical documentation.

FR-05 Manage repositories.

FR-06 Support pull request workflows.

FR-07 Generate engineering analytics.

FR-08 Execute plugins.

FR-09 Integrate CI/CD pipeline information.

FR-10 Record system logs.

FR-11 Validate user operations.

FR-12 Support optional AI-assisted functionality.

---

5. Non-Functional Requirements

Performance

The platform should efficiently support large development teams and multiple concurrent users.

Scalability

The architecture should support independently scalable services and plugin expansion.

Reliability

The platform should maintain consistent project, documentation, and repository data.

Availability

Core services should remain available during continuous software development activities.

Maintainability

The application should follow Clean Architecture, SOLID principles, and modular design.

Extensibility

New plugins and integrations should be added without modifying the core platform.

Security

Only authorized users should access projects, repositories, and documentation.

Observability

Application metrics, logs, and service health should be continuously monitored.

---

6. Assumptions and Constraints

Assumptions

- Every user has a unique account.
- Every workspace has a unique identifier.
- Every project belongs to a workspace.
- Plugin interfaces follow predefined contracts.
- PostgreSQL, Redis, Kafka, Docker, and Kubernetes are properly configured.

Constraints

- Java 21+
- Spring Boot
- PostgreSQL
- Apache Kafka
- Redis
- Elasticsearch
- Docker
- Kubernetes
- Maven or Gradle
- Clean Architecture
- Microservices Architecture

---

7. Technologies & Skills Required

Technology / Concept	Application in the Project
Java	Core language for backend development.
Spring Boot	Develop modular microservices.
Microservices Architecture	Build independently deployable backend services.
REST APIs	Enable communication between clients and services.
Apache Kafka	Support asynchronous event-driven workflows.
PostgreSQL	Store users, projects, repositories, documentation, and analytics data.
Redis	Cache frequently accessed information and active sessions.
Elasticsearch	Provide powerful searching across documentation and repositories.
Docker	Containerize backend services.
Kubernetes	Orchestrate deployment, scaling, and service recovery.
Plugin Architecture	Enable extensible modules without modifying the core application.
CI/CD Concepts	Integrate build, test, and deployment pipelines.
Design Patterns	Apply Factory, Strategy, Observer, Command, Builder, Repository, Dependency Injection, and Plugin patterns.
SOLID Principles	Build maintainable, extensible, and loosely coupled services.
Clean Architecture	Separate business logic from infrastructure and framework-specific code.
Logging & Monitoring	Record platform activities and monitor distributed services.
Unit & Integration Testing	Verify business logic, plugins, and inter-service communication.

---

8. Learning Outcomes

After completing this project, the learner will be able to:

- Design enterprise-scale software engineering platforms.
- Build production-ready Spring Boot microservices.
- Apply Clean Architecture and SOLID principles across large systems.
- Implement extensible plugin-based software.
- Design complete code review and project management workflows.
- Integrate distributed services using Kafka and REST APIs.
- Build scalable cloud-native applications with Docker and Kubernetes.
- Design analytics dashboards for engineering productivity.
- Understand enterprise DevOps workflows and CI/CD integration.

- Gain practical experience with architecture, scalability, extensibility, and engineering practices expected of senior software engineers and software architects.

Conclusion

The **Developer Productivity Platform (Notion + GitHub + CI/CD)** is the ultimate capstone project in this roadmap. It combines project management, technical documentation, repository management, code review workflows, analytics, plugin architecture, cloud-native deployment, and optional AI-assisted capabilities into a single enterprise platform. By integrating Spring Boot, Microservices, PostgreSQL, Kafka, Redis, Elasticsearch, Docker, Kubernetes, Clean Architecture, SOLID Principles, Design Patterns, and DevOps concepts, learners gain comprehensive experience in building production-grade software systems. This project demonstrates the architectural thinking, scalability, extensibility, and engineering discipline expected of senior backend engineers, technical leads, and software architects.

Documentation (The Secret Weapon)

This is the part almost nobody does.

For **every project**, create a dedicated folder like this:

```
Project-07-Expense-Tracker/
│
├── README.md
├── Requirements.md
├── Design.md
├── Architecture.md
├── Learning Journal.md
├── Challenges.md
├── Bugs.md
├── Screenshots/
├── UML/
├── Source/
└── Demo/
```

Inside your documentation, capture:

- **Project Overview** – What problem does it solve?
- **Objectives** – What did you want to learn?
- **Features** – Current functionality.
- **Technologies Used** – Java version, libraries, tools.
- **Folder Structure** – Explain how the project is organized.
- **Requirements** – Functional and non-functional requirements.

- **Architecture** – Diagrams and design decisions.
- **Class Diagram / Sequence Diagram / Use Case Diagram** – As appropriate.
- **Development Timeline** – Milestones and progress.
- **Learning Journal** – What you learned each day.
- **Challenges Faced** – Where you got stuck.
- **Debugging Stories** – How you diagnosed and fixed issues.
- **Mistakes & Lessons Learned** – What you'd do differently next time.
- **Refactoring Notes** – Improvements made after the first version.
- **Future Enhancements** – Ideas for the next iteration.

This transforms a simple coding project into an engineering case study.